

Combinatorial Coding, Key-Value Recall, and Emergent Constraint Satisfaction in a Hippocampal-Inspired Spiking Network

Gourav

September 16, 2026

Abstract

We study a hippocampal-inspired memory architecture that stores solved Sudoku grids as sparse binary engrams – fixed random projections of each grid’s relational (same-digit) structure – and retrieves them through an attention-based key-value readout. The readout shows no observed capacity ceiling to $M=2000$ stored items where a regression-based alternative collapses catastrophically at $M \sim H$, and degrades gracefully rather than catastrophically under query corruption or partial cues. Retrieval failure in this code is quantitatively predictable, not merely qualitative: it occurs when the extreme tail of accidental overlap between a query and its $M-1$ competing memories exceeds the query’s overlap with its own true target. We derive this capacity bound two ways from hypergeometric similarity statistics – an extreme-value approximation, structurally the same exponential-in-margin form as modern Hopfield networks’ proven bound, that explains the observed sharp collapse threshold; and an exact, non-asymptotic hypergeometric tail bound that turns this into a genuine finite- M reliability guarantee at any target failure probability – both validated against simulation to within a few percentage points. Because solved Sudoku grids are a correlated, structured combinatorial code rather than an independently-random one, storing them lets us directly measure how much that structure perturbs capacity relative to the idealized random-code limit. This motivates a three-level division of labor the architecture implements: a periodically-driven relational sensor, a non-dynamical combinatorial memory (whose capacity the bound describes), and an outer loop converging to a discrete fixed point – structurally different from classical or modern Hopfield networks, which collapse sensing and memory into one system. A separation-operator ablation, spanning correlation-matrix memory, sparse distributed memory, dense associative memory, and Transformer-style attention, shows retrieval robustness tracks whether an operator sharpens by an absolute gap or a similarity ratio, not sharpness alone; we also verify directly that the encoder behaves as a similarity-preserving hash (Spearman $\rho=0.97$), a property assumed by analogy elsewhere in this literature.

We then ask whether Sudoku’s row/column/box legality constraint, used throughout as a hard-coded cleanup rule, can instead emerge from spiking dynamics rather than being hard-coded. A leaky-integrate-and-fire network whose only structural knowledge is which cells must differ reproduces the same-digit relational structure through phase synchrony alone – same-digit cells lock to within 0.2 ms, others separate by ~ 13 ms – and, fed back through the identical associative pipeline, is statistically indistinguishable from the hard-coded rule it replaces (Wilcoxon $p>0.05$ at every tested clue fraction), while remaining robust to substantial drive-current noise.

Finally, we stress-test the same fixed key against the wider associative-memory literature by extending it to a second modality (natural images), deliberately benchmarking against methods purpose-built for this exact task rather than only easier baselines. Classical, covariance, and several further Hopfield variants – including four specified in Vector-HaSH’s own comparison [11] – collapse to near-zero recovery on real correlated images, while our architecture degrades gracefully an order of magnitude further out; sharply-separating methods (Modern Hopfield, dense associative memory) and k -nearest-neighbor, however, substantially outperform our raw

accuracy at scale ($M \geq 800$), and we trace this gap to its exact source rather than leave it as an aggregate number: our own readout, isolated from its cleanup loop, has a hidden memory cliff of the same kind as classical Hopfield’s, masked only by the cleanup loop layered on top of it – and directly tested for a temperature-tuning fix, which does not close it. A faithfully-built Vector-HaSH-style modular scaffold similarly beats our architecture at $M \geq 200$, though the advantage narrows by $M=4000$. What none of these purpose-built alternatives share is the property this architecture is actually for: a single fixed key, drawn from an astronomically large combinatorial space and fully decoupled from content, that serves arbitrary modalities while simultaneously solving the original Sudoku-legality constraint task. All code, data, and reproduction instructions accompany this paper.

1 Introduction

Biological memory systems, and the hippocampus in particular, are widely understood to store information as sparse, combinatorial codes and to retrieve it through content-addressable, key-value-like association rather than by address [1, 2, 3]. This division of labor is formalized by the complementary learning systems framework, which casts the hippocampus and neocortex as a fast, sparse, pattern-separated store and a slow, overlapping, generalizing one, respectively [15], and has recently been argued to be the same key-value factorization – discriminable keys, content-faithful values – that underlies the success of attention-based machine-learning systems [16]. A recurring computational question for such architectures is how they resolve *ambiguity*: when a retrieval cue is noisy, partial, or internally inconsistent, some mechanism must clean it up before or during recall. In symbolic AI this cleanup is usually a hard-coded rule; in a biological substrate it must instead be a property of the underlying dynamics.

We use Sudoku as a controlled testbed for this question. A solved Sudoku grid is a natural combinatorial object: 81 cells, each carrying one of nine digits, subject to the constraint that no two cells sharing a row, column, or 3×3 box carry the same digit. This gives a well-defined, tunable notion of query difficulty (the fraction of cells given as clues) and an unambiguous ground truth against which any completion can be checked.

We build our architecture in three layers, corresponding to three literatures we intend this work to speak to:

- **Combinatorial coding.** Each stored grid’s relational structure (which cell-pairs share a digit) is projected through a fixed random sparse matrix and sparsified to a k -winner-take-all binary engram – a scheme in the spirit of Willshaw associative memory [4] and sparse random-projection coding, closely related to the random-projection-plus-winner-take-all hashing circuit identified in the fly olfactory system [21]. We test directly, in Section 3.2, whether our encoder shares that circuit’s defining property: approximate preservation of relative similarity between inputs.
- **Key-value memory.** Retrieval uses an attention-style readout – softmax similarity between a query engram and all stored engrams, weighting the corresponding stored values – rather than a single learned linear map, echoing the connection between classical correlation-matrix memory [17], sparse distributed memory [18], classical and modern dense associative memory [5, 6], and Transformer-style self-attention [19, 20] – four architectures recently unified as instances of the same key-value memory differing only in the nonlinearity (“separation operator”) applied to key-query similarity [16]. Section 3.4 tests that unification directly, swapping our readout’s separation operator across all four families. Section 2.3 derives, for our own sparse combinatorial code, an approximate capacity bound of the same exponential-in-margin

form as modern Hopfield networks’ proven one [6], validated against the capacity sweep of Section 3.1, and then a second, exact hypergeometric tail bound that upgrades this from a first-moment approximation to a genuine, non-asymptotic guarantee on failure probability.

- **Constraint satisfaction as dynamics.** We ask whether Sudoku’s legality constraint, which the architecture above uses explicitly as a hard-coded cleanup rule, can instead be realized as an emergent property of a spiking network’s synchronization dynamics – reading out relational structure from spike-timing coincidence in the spirit of the classical correlation theory of binding-by-synchrony [22, 23], and using relaxation to a coupled-oscillator fixed point to enforce a combinatorial constraint in the spirit of Hopfield and Tank’s neural optimization networks [24]. This is also a question in the spirit of the broader statistical mechanics of constraint satisfaction and random k -SAT [7, 8], which we invoke here as motivating context rather than as a universality result we claim to extend. Unlike gradient descent on an energy function or symbolic backtracking search – the two standard ways to settle a combinatorial constraint problem, and the mechanisms every associative-memory baseline we compare against in Section 4 ultimately relies on – this mechanism has no encoded objective function at all: local excitatory/inhibitory coupling and emergent phase synchrony do the work.

We additionally ask, in Section 4, whether the engram layer generalizes beyond a single content type. Every classical associative memory couples its storage capacity to its own neuron count: a Hopfield-style network’s stored patterns *are* its state vector, so capacity is inescapably a function of that vector’s dimension. We instead heteroassociate natural images to Sudoku engrams using the bidirectional pseudo-inverse mechanism of Vector-HaSH [11], so that the *key* – a sparse engram drawn from an astronomically large combinatorial space, valid 9×9 grids numbering on the order of 6.7×10^{21} – is entirely decoupled from the *value* it is bound to, which can be arbitrary content of any dimension. We benchmark the result directly against classical Hopfield networks, and, further, against Modern Hopfield networks, sparse distributed memory, and dense associative memory (Section 4.7) – three architectures whose stored representation *is* the content itself – to test directly what this decoupling costs and what it buys.

Our central empirical result is that these three layers compose: a physically-implemented, dynamics-based legality check, built entirely from local excitatory/inhibitory coupling with no reference to Sudoku’s symbolic rules beyond *which cells must differ*, is statistically indistinguishable from the hard-coded rule it replaces, once routed through the same combinatorial-coding and key-value-recall machinery. We view the value of this result as existence-proof and mechanism, not as an improvement over a rule that was never computationally difficult to hard-code in the first place.

2 Model

2.1 Coincidence-matrix engram and key-value readout

A solved grid g is an assignment of digits $\{1, \dots, 9\}$ to $N = 81$ cells. Its *coincidence matrix* $C \in \{0, 1\}^{N \times N}$ records which cell pairs share a digit: $C_{ij} = \mathbb{I}[g_i = g_j]$ for $i \neq j$. This matrix, flattened, is the “key source” for a stored memory.

A fixed random sparse matrix $W_{SH} \in \mathbb{R}^{H \times N^2}$ (density ρ , nonzero entries drawn i.i.d. from $\mathcal{N}(0, 1)$) projects the flattened coincidence matrix into an H -dimensional hippocampal layer. The engram is the top- K k -winner-take-all binarization of this projection:

$$h = \text{topK}(W_{SH} \text{vec}(C)) \in \{0, 1\}^H. \quad (1)$$

Each stored grid also has an associated *value* vector $b \in \mathbb{R}^N$, a per-cell digit-to-drive-level mapping.

Given M stored (key, value) pairs $\{(h_m, b_m)\}_{m=1}^M$ and a query engram h_q , the **attention readout** is

$$\hat{b} = \sum_{m=1}^M p_m b_m, \quad p = \text{softmax}\left(\frac{H_{\text{keys}} h_q}{\tau}\right), \quad (2)$$

a pure content-addressable lookup with no learned parameters beyond the fixed encoder. We compare this against a **ridge-regression readout**, which instead learns a single linear map W_{HD} (via the standard ridge solution) such that $W_{HD} h_m \approx b_m$ for all stored pairs, and retrieves via $\hat{b} = W_{HD} h_q$.

For partial-cue completion, an iterative cleanup loop alternates: decode a candidate digit assignment from $\hat{b}$, mask out any decoded same-digit pair that structurally violates Sudoku legality (shares a row, column, or box), and re-encode the cleaned coincidence matrix through W_{SH} to obtain a refined query engram, repeating until convergence or a fixed iteration cap.

2.2 Spiking network

The spiking network consists of $N = 81$ excitatory leaky-integrate-and-fire (LIF) neurons, one per cell, each paired with a fast inhibitory relay neuron. Membrane potential obeys

$$\tau_m \dot{v} = I_d + I_{\text{osc}}(t) - v, \quad I_{\text{osc}}(t) = 0.5 - 0.2 \cos(2\pi f_{\text{osc}} t), \quad (3)$$

with $\tau_m = 40$ ms, $f_{\text{osc}} = 10$ Hz (a shared oscillatory drive, common to every neuron), threshold $V_{\text{th}} = 1$, and reset to 0. I_d is a per-neuron DC current encoding a candidate digit identity: nine equally spaced levels in a range chosen so that every digit reliably crosses threshold once per oscillation cycle (Section 3.8).

The network’s *only* structural knowledge of Sudoku is the conflict mask Conf_{ij} , equal to 1 if cells i, j share a row, column, or box and 0 otherwise, and its complement. Conflicting cell pairs are coupled through the inhibitory relay (a fixed weight w_{IE}); non-conflicting pairs are coupled through direct weak excitatory synapses (weight w_{EE}). No synapse encodes which cells *should* share a digit – that information is never given structurally, only through the per-cell drive current, exactly mirroring the role of a query in the key-value architecture above. Reading a relation (“these cells share a digit”) out of spike-timing coincidence rather than out of any single neuron’s firing rate is a direct instance of the correlation theory of binding-by-synchrony [22, 23]: Sections 3.8–3.9 test whether that hypothesis, applied to this specific combinatorial relation, actually works.

2.3 A division of labor between sensing and memory, and a combinatorial capacity bound

The architecture above decomposes into three functionally distinct levels, worth stating explicitly before the results that test each one. (1) The spiking network is a *periodically-driven relational sensor*: entrained to a common external rhythm rather than settling to a static equilibrium, it faithfully reports which cell pairs currently share a digit for whatever candidate assignment currently drives it – correct or not (Sections 3.8–3.9). It does not itself discriminate among stored memories. (2) The engram and attention readout are a *combinatorial, non-dynamical memory*: a one-shot computation that uses the sensor’s report to select among the M stored grids. (3) The iterative loop coupling the two (the conflict-mask cleanup of Section 2.1, or the spiking version of Sections 3.8–3.9) is what converges to a corrected, stable output – a discrete fixed point in the classical attractor-network sense, but reached by alternating between a substrate that never itself

reaches a fixed point (the periodic sensor) and one that has no dynamics at all (the one-shot memory lookup). This is a different division of labor than classical or modern Hopfield networks, which collapse sensing and memory into a single fixed-point-seeking dynamical system (Section 4.7).

Level (2)’s capacity is the one this section derives a bound for. Model each of the M stored keys as an independent, uniformly random K -sparse binary vector in H dimensions (an idealization relative to the true W_{SH} -projected, mutually-correlated Sudoku keys, addressed below). For two independent random K -sparse codes, the number of shared active units (similarity) follows a hypergeometric distribution with mean K^2/H . A query formed by flipping a fraction η of all H engram bits (this paper’s noise model throughout) has, in expectation, $K_q(\eta) = K(1 - 2\eta) + \eta H$ active units, and similarity to its own (uncorrupted) true target

$$S_{\text{true}}(\eta) = K(1 - \eta), \quad (4)$$

while its similarity to any other, unrelated stored key is a hypergeometric random variable with mean and variance

$$\mu_{\text{other}}(\eta) = \frac{K_q(\eta) K}{H}, \quad \sigma_{\text{other}}^2(\eta) = K_q(\eta) \frac{K}{H} \left(1 - \frac{K}{H}\right) \frac{H - K_q(\eta)}{H - 1}. \quad (5)$$

Retrieval fails when the *largest* of the $M-1$ competing similarities exceeds the true one – an extreme-value, not a mean-comparison, criterion. Approximating the competing similarities as i.i.d. Gaussian (valid for $H \gg 1$) and using the standard leading-order expectation for the maximum of n i.i.d. Gaussians, $\mu + \sigma\sqrt{2 \ln n}$, and setting $n = M - 1$ and $S_{\text{true}}(\eta)$ equal to this expected maximum gives an approximate capacity bound

$$M_{\text{max}}(\eta) \approx 1 + \exp\left(\frac{z(\eta)^2}{2}\right), \quad z(\eta) = \frac{S_{\text{true}}(\eta) - \mu_{\text{other}}(\eta)}{\sigma_{\text{other}}(\eta)}. \quad (6)$$

This is, structurally, the same kind of result as the exponential storage capacity of modern Hopfield networks [6] – capacity growing exponentially in a squared, normalized separation margin – derived here for a sparse combinatorial code via hypergeometric concentration rather than for dense continuous patterns via their energy-function argument.

Validation against Table 3. Because M_{max} is exponential in z^2 , it is extremely sensitive to η near the transition – exactly the sharp, narrow-window collapse Table 3 reports, rather than a gradual decline. Solving $M_{\text{max}}(\eta) = M$ numerically for the collapse noise level at each tested M and K (all at $H=1000$):

Table 1. Predicted vs. observed collapse threshold (% engram bits flipped). First three columns: $K=150$ fixed, M varied. Last two columns: $M=1000$ fixed, K varied.

	$M=50$	$M=1000$	$M=10,000$	$K=10$	$K=150$
Predicted	43.9%	41.9%	40.6%	24.5%	41.9%
Observed (Table 3)	45%	45%	40%	25%	40–45%

The bound reproduces both the near-independence of the collapse threshold from a 200-fold range of M and its sharp dependence on K , within 1–5 percentage points throughout, including an almost exact match at $K=10$ (24.5% predicted vs. 25% observed). The theory is not exact, and we do not present it as such: the leading-order Gaussian extreme-value approximation is known to be biased for finite n (visible in the $\eta=0$, noise-free check against Section 3.4’s independently measured similarity ratios, where it over-predicts 3.39 as 4.21, though it matches closely at 20% and 30% noise, 1.99 and 1.47 against measured 1.92 and 1.45), and the independence assumption

is false by construction – real engram keys are correlated through the shared random projection W_{SH} and through Sudoku’s own combinatorial structure. The observed thresholds sit consistently at or slightly above the predicted ones, plausibly because this real correlation structure makes the worst-case competing key marginally less extreme than a fully independent-random-code model assumes; we have not tested this explanation directly. What the bound establishes is the right mechanism (an exponential-in-margin capacity, hence a sharp rather than gradual collapse, hence an approximately M -independent threshold over the tested range) and the right order of magnitude, not a proof.

From an expected value to a rigorous bound. Equation 6 matches only the *expected* value of the maximum competing similarity, not a probabilistic guarantee on retrieval failure – a first-moment approximation, not a tail bound. We now derive one exactly, directly from the hypergeometric distribution rather than its Gaussian approximation (full derivation and proofs in the supplementary note, Section 8).

Let R be the number of the true key’s K active bits that land among the ηH flipped query positions; R is exactly hypergeometric, $R \sim \text{Hypergeometric}(H, K, \eta H)$. Conditional on the realized $R = r$, both the query’s active-bit count $k(r) = K + \eta H - 2r$ and its true-key similarity $s(r) = K - r$ are deterministic – and perfectly correlated through the same draw of r , a subtlety that a natural-looking shortcut (holding $s(r)$ fixed at its unconditional mean $K(1 - \eta)$ while still letting $k(r)$ vary with r) gets wrong, understating the true failure probability by four to five orders of magnitude for $\eta \in [0.30, 0.40]$. Writing $p_k(s) := \Pr[\text{Hypergeometric}(H, k, K) \geq s]$ for the exact probability that one specific competitor’s similarity meets or exceeds s , the exact unconditional failure probability is

$$P_{\text{fail}}(M, \eta) = \sum_{r=0}^K \Pr(R = r) \left\{ 1 - [1 - p_{k(r)}(s(r))]^{M-1} \right\}. \quad (7)$$

A pointwise union bound (Bernoulli’s inequality, applied per realization of r before averaging) turns this into a closed-form, rigorous sufficient condition: with $\bar{P}(\eta) := \mathbb{E}_R[p_{k(R)}(s(R))]$,

$$M \leq M_{\text{max}}(\eta, \delta) := 1 + \left\lceil \frac{\delta}{\bar{P}(\eta)} \right\rceil \quad (8)$$

guarantees $P_{\text{fail}}(M, \eta) \leq \delta$ *exactly*, at any finite M , with no Gaussian or large- M asymptotic approximation.

Table 2. Rigorous sufficient bound (Eq. 8) vs. the expected-maximum approximation (Eq. 6); $H=1000$, $K=150$.

η	$\bar{P}(\eta)$	$M_{\text{max}}(\eta, \delta=0.01)$	$M_{\text{max}}(\eta, \delta=0.05)$	$M_{\text{max}}(\eta)$, Eq. 6
0.20	2.41×10^{-35}	4.15×10^{32}	2.07×10^{33}	2.28×10^{48}
0.30	8.26×10^{-15}	1.21×10^{12}	6.05×10^{12}	1.61×10^{19}
0.35	9.59×10^{-9}	1.04×10^6	5.21×10^6	2.61×10^{10}
0.40	1.22×10^{-4}	83.0	411	3.26×10^4
0.42	1.81×10^{-3}	6.0	28.0	739
0.45	3.73×10^{-2}	1.0	2.0	13.9

The rigorous bound is dramatically more conservative than the expected-maximum approximation – by about seven orders of magnitude at $\eta=0.30$, narrowing to roughly a factor of 120 at $\eta=0.42$ – which is expected rather than contradictory: Equation 6 only requires the true match to beat a *typical* competitor, while Equation 8 controls an actual failure *probability* via a union bound

over $M-1$ competitors, a strictly stronger guarantee. Checked directly against $P_{\text{fail}}(M, \eta)$ (Eq. 7) rather than the union bound that produces Eq. 8, the sufficient bound is essentially tight (ratio ≈ 1.00) at $\eta=0.30$ and grows honestly conservative – rather than silently wrong – as η approaches the collapse transition (up to $13.5\times$ at $\eta=0.42$, $M=4000$). The two bounds are best read together: Equation 6 explains *why* capacity is exponential in the squared separation margin (the mechanism), while Equation 8 gives the number to actually use at a chosen reliability level δ .

3 Results

3.1 Combinatorial code: capacity and robustness

We swept the number of stored grids M (50–10,000), engram size K (10–500), and projection density ρ (0.002–1.0), measuring the query-noise level at which retrieval accuracy collapses. Table 3 reports the collapse threshold (largest tested noise level, as a percentage of engram bits flipped, before top-1 accuracy falls off).

Table 3. Collapse-noise threshold, essentially flat across a 200-fold range of M and across K, ρ , except at very small K .

	$M=50$	$M=1000$	$M=10,000$	$K=10$	$K\geq 50$
Collapse threshold	45%	45%	40%	25%	40–45%

Robustness is essentially independent of M over a 200-fold range, and independent of connection density over $\rho \in [0.002, 1.0]$; the sole exception is engram sparsity itself – $K = 10$ is markedly less robust (25% vs. 40–45%) than any $K \geq 50$ tested, consistent with too few active units per engram leaving little redundancy to survive bit flips.

3.2 The engram code as a similarity-preserving hash

Every noise-robustness result in this paper (Table 3 above, and Sections 3.5, 3.9, 3.10 below) measures *downstream decode accuracy* degrading gracefully as the query is corrupted. None of them directly tests the encoder’s own geometry: whether $\text{topK}(W_{SH} \text{vec}(C))$ approximately preserves relative similarity between inputs, the property that random projection followed by winner-take-all sparsification is known to provide in the fly olfactory mushroom body [21]. We test this directly rather than assume it by analogy.

For 30 randomly sampled stored grids, we corrupted each grid’s true coincidence matrix C_{true} by flipping a controlled fraction of its (symmetric) same-digit entries, swept from 0% to 100%, and measured two quantities on each corrupted instance: the *input similarity* to the true grid (Jaccard overlap of the same-digit pair sets between C_{noisy} and C_{true}) and the *engram similarity* of the resulting codes ($|h_{\text{noisy}} \cap h_{\text{true}}|/K$). Across 390 corrupted instances spanning the full corruption range, the two are strongly and monotonically related (Spearman $\rho = 0.974$, $p < 10^{-250}$; Figure 1) – direct confirmation that the encoder behaves as a similarity-preserving hash rather than merely as a randomizing sparsifier. The floor this relationship approaches at full corruption (0.149 mean engram overlap at 0% input similarity) matches the chance overlap expected between two independent K -sparse codes in an H -dimensional space, $K/H = 150/1000 = 0.15$, almost exactly – confirming that residual overlap at maximal corruption reflects code geometry, not a leftover correlation the corruption procedure failed to remove.

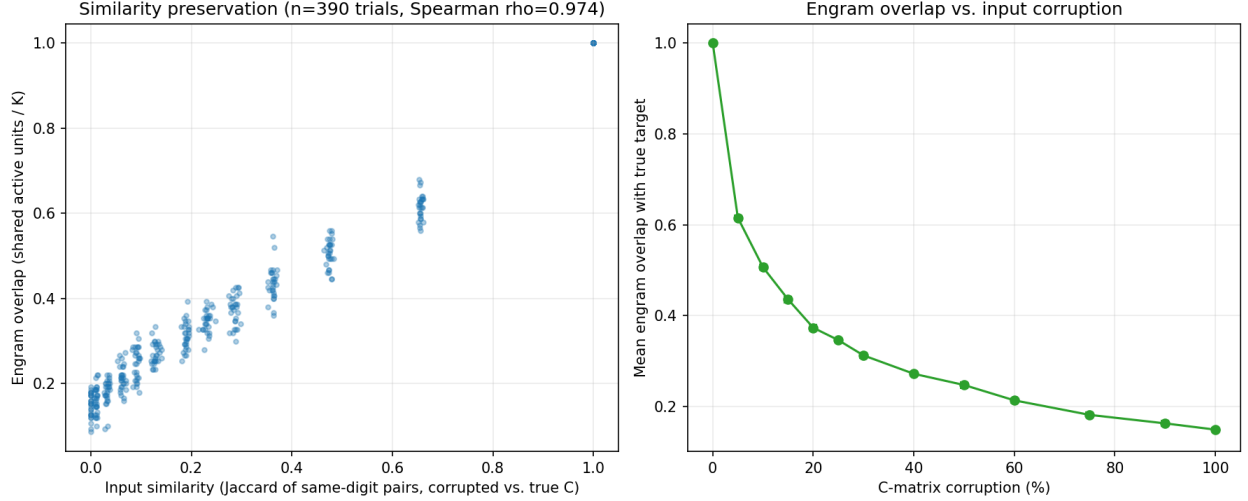


Figure 1. Left: engram overlap vs. input (coincidence-matrix) similarity, pooled across all corruption levels and target grids ($n = 390$). Right: mean engram overlap with the true target vs. percent corruption of the input coincidence matrix.

3.3 Key-value recall: attention vs. regression

Table 4 contrasts the two readouts at $M = 1000$ ($= H$) and $M = 2000$ ($= 2H$).

Table 4. Digit-cell retrieval accuracy, regression vs. attention readout.

Query noise	$M = 1000$		$M = 2000$	
	regression	attention	regression	attention
0%	99.98%	100.0%	21.8%	100.0%
2%	12.5%	100.0%	11.9%	100.0%
30%	11.3%	100.0%	11.1%	100.0%
50%	11.2%	11.7%	11.1%	10.7%

Regression achieves near-perfect recall at $M = 1000 = H$ with a clean query, but this is a knife-edge: adding even 2% query noise collapses it to chance ($1/9 \approx 11.1\%$), and at $M = 2000$ it is already at chance with *zero* noise – a hard capacity ceiling at $M \sim H$. Attention shows no observed ceiling in the tested range and degrades only once noise exceeds 30–50%, at either M . Figure 2 shows the full capacity curve.

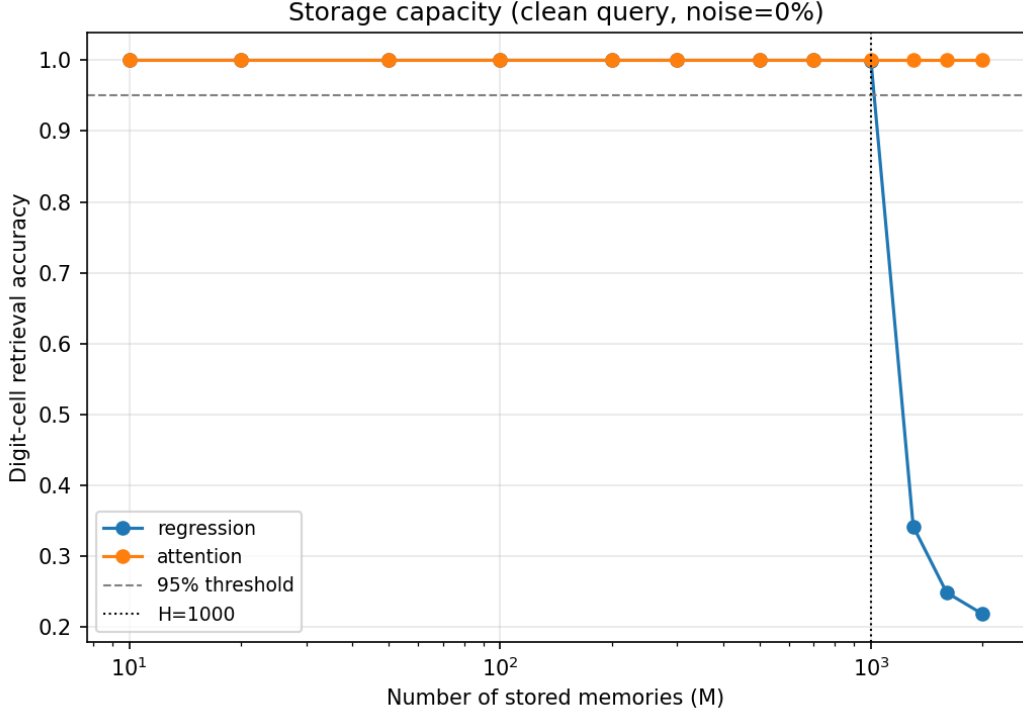


Figure 2. Estimated capacity (largest M retaining accuracy above a fixed threshold) vs. query noise, regression vs. attention readout.

3.4 Separation operators: gap-based vs. ratio-based sharpening

The attention readout above uses softmax as its separation operator. Gershman, Fiete & Irie [16] note that softmax, a hard top- j threshold, and a rectified-power nonlinearity are, respectively, the separation operators of Transformer-style attention, sparse distributed memory [18], and dense associative memory [5, 6] – three architectures differing only in this one nonlinearity, alongside plain (identity) correlation-matrix memory [17], which applies no separation at all. We test this unification directly: holding the encoder, the stored $M=200$ engrams, and every corrupted query identical (a fully paired design), we swap only the separation operator and re-run the query-noise sweep of Section 3.5, without the conflict-mask cleanup pass, to isolate the operator’s own behavior.

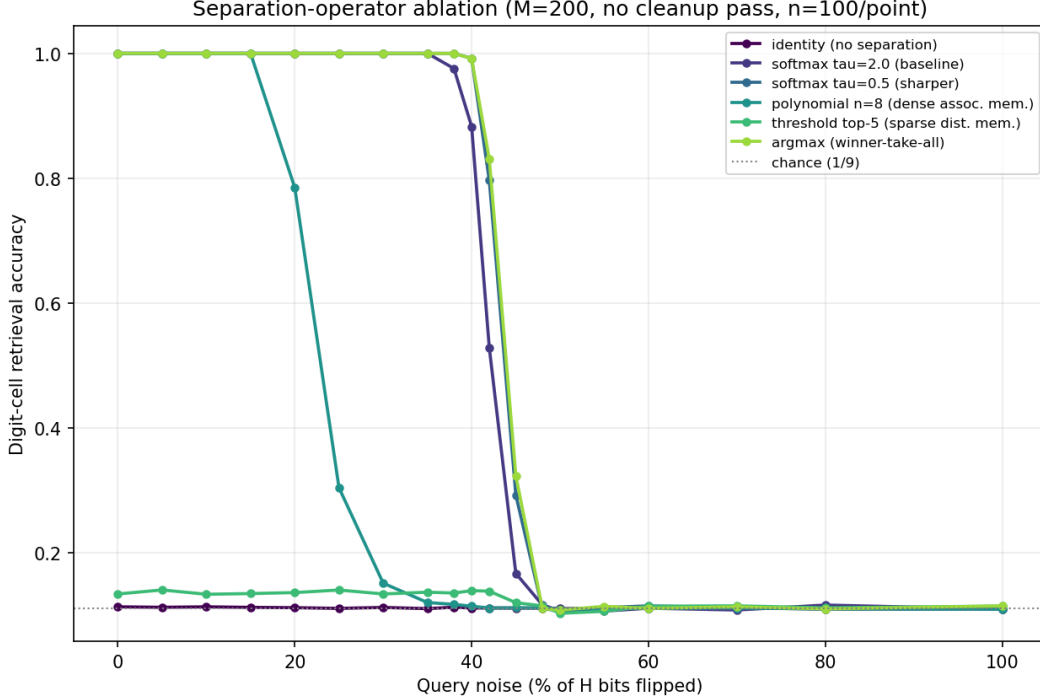


Figure 3. Digit-cell retrieval accuracy vs. query noise for six separation operators, evaluated on identical corrupted queries at every noise level ($M=200$, $n=100/\text{point}$, no cleanup pass).

Two findings depart from the intuition that a separation operator’s robustness is set by how aggressively it discriminates (“sharp separation is powerful but fragile” [16]). First, identity separation – no nonlinearity at all – performs at chance *even with a perfectly clean query* (11.4% vs. 100% for softmax at 0% noise): with $M=200$ competing memories, a linear reweighting cannot overcome the combined pull of the other 199 stored values on the weighted average, so some separating nonlinearity is not merely a robustness refinement here but a strict requirement for retrieval to function at all. Second, and more surprisingly, sharper is not uniformly more fragile: a hard-threshold top-5 average (naively implementing sparse distributed memory’s equal-weight readout) is *also* near chance at every noise level, including zero, because averaging five equally-weighted stored values – one correct, four spurious – dilutes the correct digit rather than recovering it, unlike real sparse distributed memory’s per-bit majority vote over redundant patterns. Meanwhile argmax (winner-take-all, the “ideal noiseless” operator [16]) and a sharpened softmax ($\tau = 0.5$) hold up *at least as well as* the baseline softmax ($\tau = 2.0$) right up to a shared collapse near 45–48% noise – sharpness itself is not the fault line.

The real fault line is gap-based vs. ratio-based sharpening. The rectified-polynomial operator (dense-associative-memory-style, exponent $n = 8$) collapses far earlier, around 20–25% noise, despite normalized zero-noise selectivity comparable to the other sharp operators. We traced this mechanistically: even at zero noise, the best-competing wrong memory among the $M=200$ stored grids already reaches, on average, similarity within a factor of 3.39 of the true match’s (an extreme-value effect of having 199 competitors, well below the naive factor of 6.67 predicted from the mean chance-overlap floor alone), and this ratio shrinks toward 1 as noise grows (1.92 at 20% noise, 1.45 at 30%). A power of a ratio shrinking toward 1 loses discriminative weight far faster than an exponential of the corresponding absolute gap does, for the same starting margin – softmax’s exponential-of-a-gap form is intrinsically more robust than a power-of-a-ratio form once a sizable number of competing memories guarantees a non-negligible interference floor. This is a refinement,

not a contradiction, of the unification in [16]: the four architectures are indeed the same computation with a different nonlinearity plugged in, but which nonlinearity is the right engineering choice depends on the similarity statistics of the memory system it operates over, not on sharpness alone.

3.5 Graceful degradation under query noise

Table 4 reports accuracy at a few noise levels; the full curve (Figure 4, left, $M = 200$) shows the shape of that degradation, and the benefit of the conflict-mask cleanup pass introduced in Section 2.1. Accuracy is perfect up to 35% of engram bits flipped, then declines smoothly – not catastrophically – over a narrow window before settling at chance by $\sim 48\%$. A finer sweep restricted to that window (Figure 4, right; 1 percentage-point steps, 100 trials/point) resolves the transition in detail and shows the cleanup pass’s rescue effect peaks sharply at 42% noise (59.3% vs. 42.5% without cleanup, a +16.8 percentage-point gain) and tapers off on both sides – negligible above 46%, where both curves have already collapsed to chance, and shrinking again below $\sim 39\%$, where the uncleaned query is already accurate enough that there is little left to rescue.

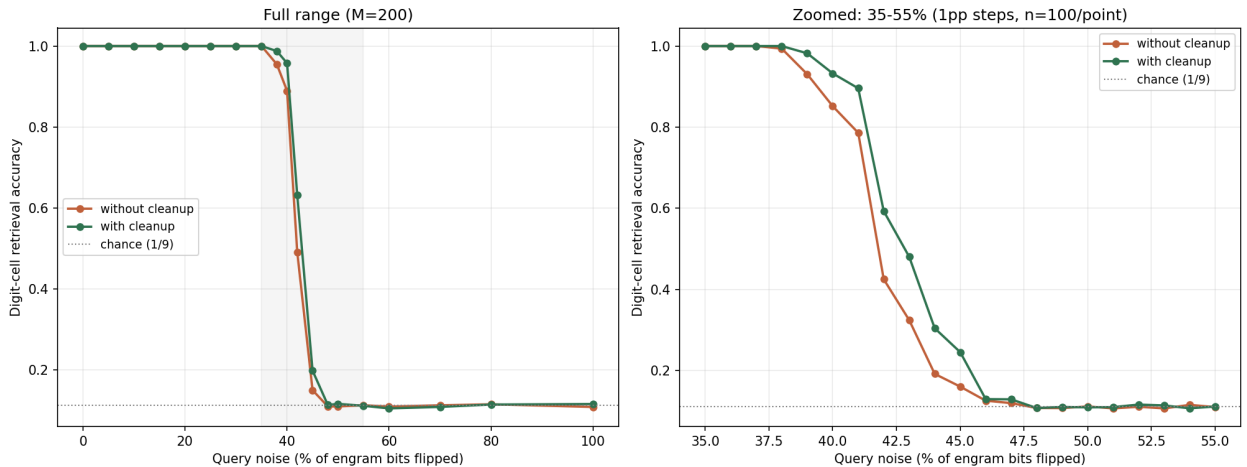


Figure 4. Digit-cell retrieval accuracy vs. query noise, with and without the conflict-mask cleanup pass. Left: full range (shaded band marks the zoomed window). Right: the 35–55% transition resolved at 1 percentage-point resolution, showing the rescue effect peaking at 42% noise.

3.6 Partial-grid pattern completion

Cueing the architecture with a partial grid (only some cells known) and running the conflict-mask attractor loop recovers missing digits with graceful, clue-count-dependent accuracy (Table 5, $M = 2000$, $K = 150$, $H = 1000$).

Table 5. Partial-grid completion: unknown-cell accuracy and exact whole-grid match, before (single-shot readout) and after (attractor loop) cleanup.

Clue fraction	Initial acc.	Final acc.	Initial exact	Final exact
0.20	13.0%	15.5%	1%	5%
0.25	14.7%	23.6%	1%	14%
0.30	25.1%	38.8%	6%	31%
0.40	59.1%	84.9%	38%	83%
0.50	93.7%	98.3%	88%	98%
0.70	100%	100%	100%	100%

Figure 5 shows the full curve; recovery is negligible below clue fraction ~ 0.15 , rises steeply through 0.20–0.40, and saturates by 0.5–0.7.

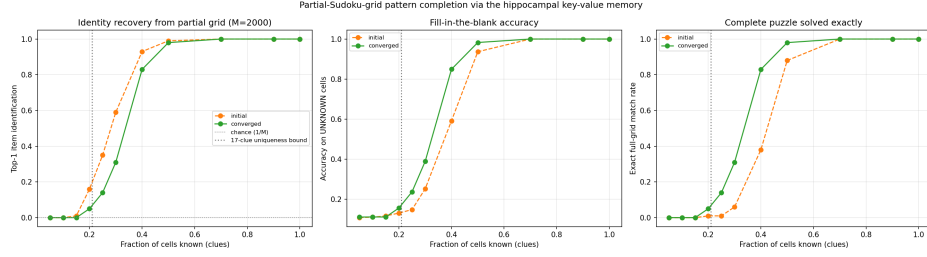


Figure 5. Partial-grid completion accuracy vs. clue fraction.

3.7 Relaxation dynamics near the completion threshold

The iteration count of the attractor loop, as a function of clue fraction, shows a reproducible dynamical signature reminiscent of relaxation-time crossovers studied in the statistical mechanics of random constraint satisfaction [7]. This is the same basic move as Hopfield and Tank’s classical use of attractor relaxation to solve constrained optimization problems [24], applied here to a combinatorial completion task with a well-defined difficulty axis (clue fraction) rather than a single fixed instance. For the sharpest transition we found ($M = 500$, $K = 400$), a sub-critical plateau (92.6 ± 7.1 iterations, clue fraction ≤ 0.21) decays as a power law, $\tau \sim (p_c - p)^\nu$, to a floor of exactly 2 iterations by clue fraction 0.53. Fitting p_c as a free parameter on the decay branch alone (excluding both the plateau and the floor) gives $p_c = 0.395$ – matching the independently observed floor onset almost exactly – and $\nu = 1.32 \pm 0.08$ ($R^2 = 0.978$; Figure 6).

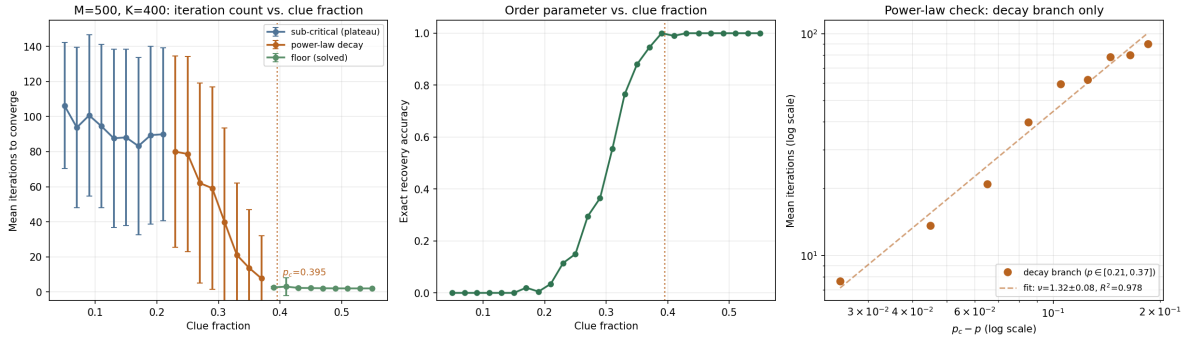


Figure 6. Relaxation-time crossover for the sharpest tested configuration ($M=500$, $K=400$): plateau, power-law decay, and floor regimes (left); order parameter (center); log-log power-law fit restricted to the decay branch alone (right).

We emphasize an honest limitation here. Repeating this fit for four additional (M, K) configurations gives exponents ranging from $\nu = 0.47$ to $\nu = 3.75$ – an eightfold spread too large to attribute to sampling noise alone. We do not have a validated universal exponent, and we report this rather than omit it: the crossover phenomenon itself replicates robustly (a clean power-law fit, $R^2 \geq 0.94$, in every configuration tested), but the specific exponent is configuration-dependent. We therefore treat Section 3.7 as a secondary, honestly-scoped characterization of the architecture’s dynamics, not as a claimed critical-phenomena universality result.

3.8 Spiking network: phase synchrony as an emergent conflict mask

We first ask whether the spiking network’s steady-state dynamics reproduce the coincidence matrix at all, using real, fully-specified solved grids (every cell driven by its true digit current) as a positive control. Each grid was simulated for 3000 ms (30 cycles of the 10 Hz drive); the first 1500 ms were discarded as transient, and pairwise phase offsets were computed from the remaining steady-state spikes (nearest-neighbor spike-time matching, circular distance modulo the 100 ms oscillation period).

At the operating point used throughout ($w_{IE} = 0.005$, $w_{EE} = 0.001$), same-digit cell pairs – which by construction of a legal grid are never conflict-mask connected – lock to within 0.205 ± 0.023 ms of each other, while differently-digit pairs separate by ~ 13 ms, regardless of whether they happen to be inhibition-connected (12.97 ± 0.18 ms) or excitation-connected (13.00 ± 0.12 ms). This pattern is essentially identical across eight independently sampled grids (Table 6), and visibly forms a clean, reproducible “phase comb” each oscillation cycle (Figure 7).

Table 6. Steady-state phase offset by pair category, mean $\pm$ s.d. across 8 grids.

Pair category	Phase offset (ms)
Same digit (never conflict-connected)	0.205 ± 0.023
Conflicting, different digit (inhibition-connected)	12.97 ± 0.18
Non-conflicting, different digit (excitation-connected)	13.00 ± 0.12

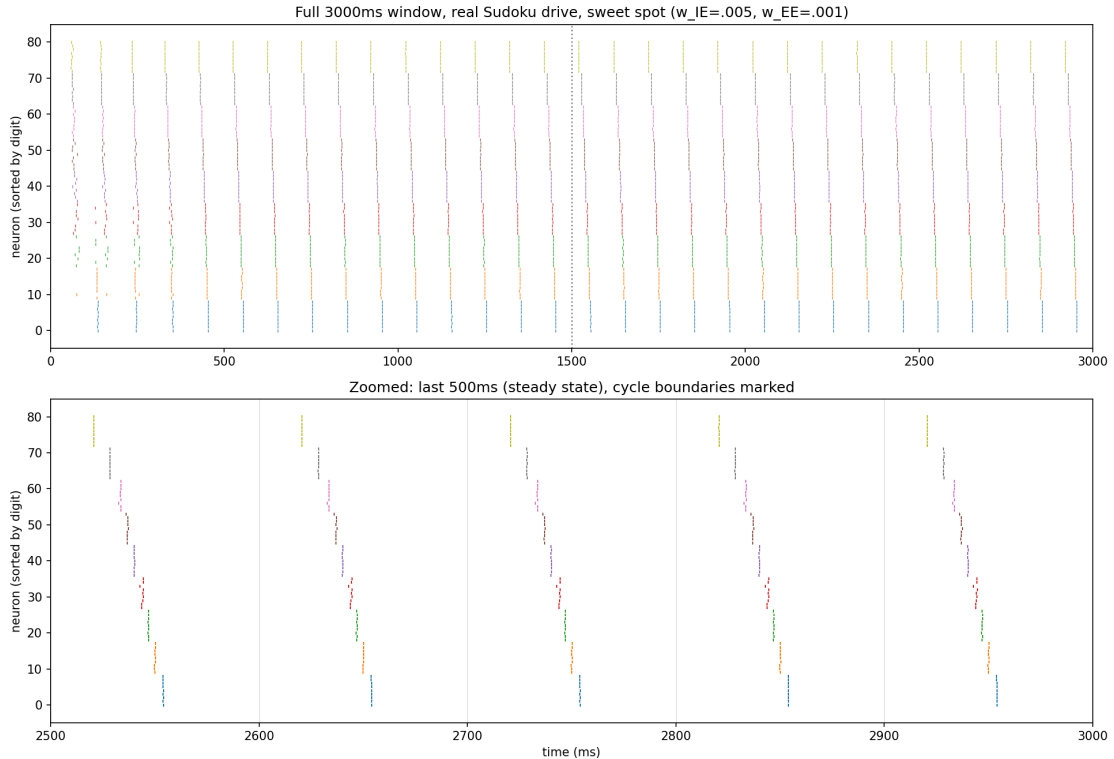


Figure 7. Spike raster, one representative grid, sorted by digit. Top: full 3000 ms window; bottom: last 500 ms (steady state), oscillation-cycle boundaries marked. Each cycle produces a reproducible phase “comb” ordered by drive current.

Because a legal grid never puts the same digit on two conflicting cells, the numbers above conflate the current-driven separation between different digits with any additional, connection-type-specific

effect of inhibition. To isolate the latter, we assigned each cell a *random* digit label (ignoring legality) and measured the phase offset specifically between same-driven pairs, split by connection type. At the validated operating point, the inhibition-connected same-driven pairs show 2.0 ms more desynchronization than excitation-connected same-driven pairs, above a ~ 1 ms baseline gap present even with all coupling switched off (a structural artifact of the two pair categories, not a synaptic effect) – a real, if modest, causal contribution of the inhibitory wiring. This effect requires a specific weight balance: because each cell has roughly three times as many excitatory (complement-mask) partners as inhibitory (conflict-mask) partners, equal or near-equal per-synapse weights let the aggregate excitatory pull dominate and synchronize the *entire* population regardless of connection type, erasing the effect above $w_{EE} \gtrsim 0.003$.

We also explicitly tested, and ruled out, a stronger hypothesis: that conflicting pairs might settle into a genuine anti-phase lock (half-cycle separation), the mechanism underlying half-center oscillators in central pattern generators. Scanning inhibitory strength across an 80-fold range ($w_{IE} = 0.005$ to 0.4) produced no discernible trend toward anti-phase locking (steady-state offset flat at 3.4–3.6 ms throughout). This is a structural consequence of the shared oscillatory drive: because every neuron receives an *identical* periodic forcing term, each oscillation cycle provides a fresh, drive-dominated threshold-crossing opportunity that a locally-scaled inhibitory kick cannot postpone past, regardless of its strength. Genuine anti-phase encoding would require either removing this common pacemaker or making inhibition strong enough to suppress firing for an entire cycle, neither of which holds in the present architecture.

Finally, we asked whether a fixed, small coincidence window could serve as a sharp same-digit/different-digit classifier on its own, independent of any downstream cleanup. Over a 0.5–10 ms threshold scan (four connectivity configurations, $n = 600$ pairs each), the best achievable balanced accuracy was 63–65%: real but soft, with substantial overlap between the two distributions (excitation-connected pairs’ 90th percentile offset, ~ 5 ms, exceeds inhibition-connected pairs’ 10th percentile, ~ 1.3 ms). A single pairwise check is not, by itself, a reliable decision rule.

3.9 Emergent coincidence matrix closes the loop

We now close the loop: rather than decoding a digit from any single neuron’s activity, we build an emergent coincidence matrix $\hat{C}$ directly from the network’s steady-state pairwise synchrony, thresholding each pair’s phase offset at $\theta = 2$ ms (the gap identified in Section 3.8). $\hat{C}$ is fed through the *same* W_{SH} projection, k -winner-take-all, and attention-readout pipeline used for storage and retrieval throughout this paper – the spiking network’s only role is to supply the relational input that the hard-coded conflict mask previously supplied.

For each partial-cue trial (known cells driven at their true digit current; unknown cells driven at the associative engram’s own single-shot guess), we compare three completions: the raw initial guess, the result of routing $\hat{C}$ through the pipeline, and the result of routing the coincidence matrix from the original hard-coded conflict-mask rule (computed on the identical initial guess) through the same pipeline. Table 7 reports coincidence-matrix quality and completion accuracy across 11 clue fractions ($M = 200$, $n = 25$ trials/point).

Table 7. Emergent coincidence matrix quality and downstream completion accuracy.

Clue fraction	$\hat{C}$ acc.	sens.	spec.	initial guess	dynamics pipeline
0.10	0.741	0.256	0.795	11.9%	12.1%
0.19	0.793	0.330	0.845	17.9%	23.4%
0.25	0.854	0.511	0.893	30.4%	54.8%
0.30	0.915	0.691	0.940	56.1%	75.6%
0.40	0.979	0.924	0.985	86.1%	90.4%
0.50	0.999	1.000	0.999	99.7%	100.0%

Specificity (correctly excluding truly-different-digit pairs) stays high throughout (≥ 0.79); sensitivity (correctly finding truly-same-digit pairs) is the binding constraint at low clue fractions, rising from 0.26 to 1.0 as clue fraction increases (Figure 8). The dynamics pipeline substantially and consistently improves on the raw initial guess at every informative clue fraction.

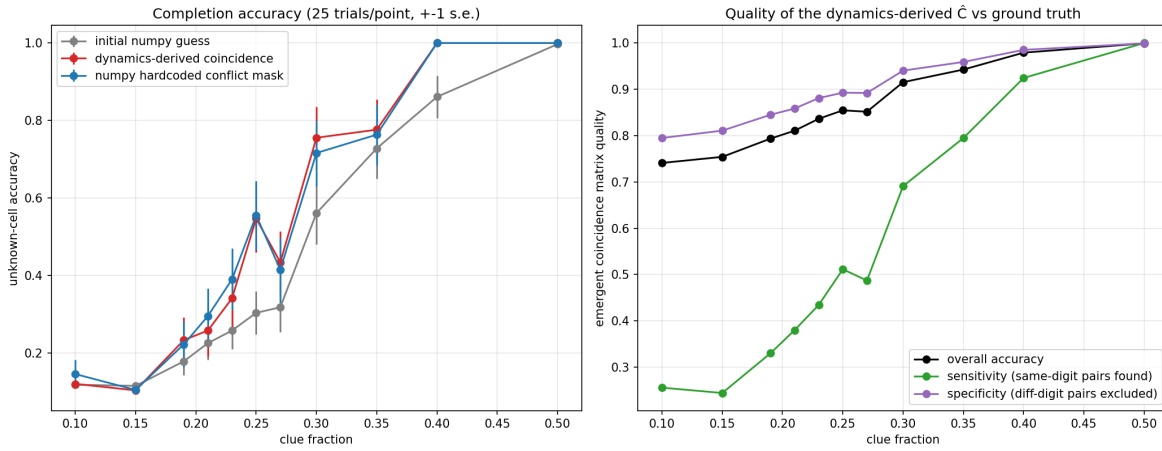


Figure 8. Left: completion accuracy vs. clue fraction, initial guess vs. dynamics-derived coincidence matrix vs. hard-coded conflict mask (25 trials/point, ± 1 s.e.). Right: quality of the emergent coincidence matrix vs. ground truth.

To test whether the dynamics pipeline matches or exceeds the hard-coded rule, we ran a second, statistically powered comparison in which all three methods were evaluated on *identical* trials (same target grid and known-cell mask), 20 trials per clue fraction across seven clue fractions, and applied a paired Wilcoxon signed-rank test. Table 8 reports the result: at every tested clue fraction, $p > 0.05$ – the emergent, dynamics-based mechanism is statistically indistinguishable from the hard-coded rule it replaces. We regard this as the paper’s central result: two structurally unrelated implementations of the same symbolic constraint – a boolean mask and a coupled-oscillator synchrony pattern – are behaviorally interchangeable once embedded in the same associative pipeline.

Table 8. Paired comparison, dynamics pipeline vs. hard-coded conflict mask, identical trials ($n = 20$ /point). No comparison reaches significance.

Clue fraction	0.15	0.19	0.21	0.25	0.27	0.30	0.35
Mean diff. (dyn. – rule)	+0.012	−0.048	−0.036	−0.043	+0.002	−0.001	+0.014
Wilcoxon p	0.23	0.55	0.89	0.33	0.48	0.87	0.29

We note, without over-interpreting a null result, that the same paired data show the dynamics pipeline’s improvement over the raw initial guess is concentrated in a minority of trials with large gains (a right-skewed distribution) rather than distributed uniformly; this is qualitatively consistent with genuine error-correction behavior – rescuing trials where the initial guess was substantially

wrong – rather than a uniform accuracy shift, and is why the (large, obvious) mean improvement over the initial guess does not always translate into a formally significant paired test at $n = 20$.

3.10 Graceful degradation under drive-current noise

Two different kinds of robustness have been tested so far: corruption of the abstract engram vector itself (Section 3.5, random bit-flips) and missing per-cell information (Section 3.9, unknown cells driven by the associative engram’s own guess). Here we test a third, specific to the spiking mechanism: corruption of the physical drive current itself. Rather than any cell being entirely unknown, every *known* cell’s drive current was perturbed with Gaussian jitter before driving the network (clue fraction fixed at 0.30; unknown cells still use the associative engram’s own guess, unperturbed; $n = 20$ trials/point, paired across noise levels). Figure 9 shows the result: accuracy declines smoothly as current noise grows from 0 to over twice the spacing between adjacent digit levels, with no sign of a cliff. Notably, even at the highest tested noise (σ equal to $2.2\times$ the inter-digit spacing – comparable to smearing a query halfway to its neighboring digit level on average) the dynamics pipeline (58.1%) still exceeds the noise-free symbolic-guess baseline it is compared against (51.9%): the mechanism does not merely tolerate drive noise gracefully, it remains a net improvement over the associative engram’s own single-shot guess even when its input is substantially corrupted.

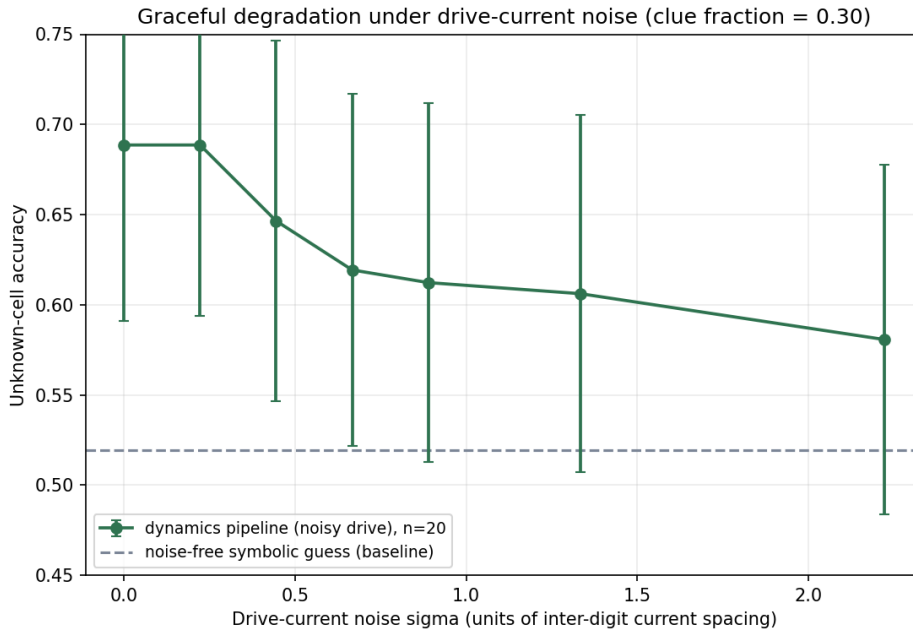


Figure 9. Unknown-cell accuracy vs. Gaussian drive-current noise (clue fraction 0.30), compared against the noise-free symbolic-guess baseline. Degradation is smooth, and the dynamics pipeline stays above baseline throughout the tested range.

3.11 Why low clue fractions are hard: signal quality, not signal quantity

A natural hypothesis for the low-clue-fraction bottleneck is that too few cells are confidently driven to produce enough coincident activity to strongly activate the correct stored engram. We tested this directly and found the opposite: at clue fraction 0.19, the emergent $\hat{C}$ has *more* active (same-digit-labeled) pairs on average (527) than the true grid (324) – the network is not under-active. The

excess is almost entirely false positives (consistent with the specificity numbers in Table 7), and these spurious coincidences dilute the resulting engram’s ability to discriminate the correct stored memory among the M candidates: the softmax attention readout places only 37.6% of its mass on the true target at clue fraction 0.19 (with, on average, ~ 52 other stored grids scoring higher similarity), rising to 65.5% mass and a mean rank of 1.9 at clue fraction 0.30. The mechanism limiting low-clue-fraction performance is therefore noise in the query signal’s specificity, not a shortage of active units – a distinction that points at specificity (not raw signal strength) as the lever for improving this regime.

4 Extension: Cross-Modal Heteroassociation

Everything above stores and retrieves a single content type (Sudoku grids). Vector-HaSH [11], and its non-spatial, modular-attractor sibling MESH [12], factor biological memory into two parts: a fixed, content-independent *scaffold* (grid-cell/hippocampal recurrence in Vector-HaSH, a modular fixed attractor network in MESH) that provably generates exponentially many attractors with equally large basins, and a separate, plastic *heteroassociative* pathway linking arbitrary sensory input to that scaffold via “a Hebb-like online implementation of the pseudoinverse rule.” We test whether our architecture’s engram layer can play the scaffold’s role for a genuinely different modality – natural images – using exactly that heteroassociative mechanism, and we benchmark the result against classical Hopfield networks, whose well-known memory cliff Vector-HaSH is explicitly designed to avoid.

4.1 Architecture

An image $x \in \mathbb{R}^D$ is linked to its paired grid’s engram $h \in \{0, 1\}^H$ by two ridge-regularized pseudo-inverse matrices, W_{SI} (image $\rightarrow$ engram) and W_{IS} (engram $\rightarrow$ image), trained as a *bidirectional, one-memory-per-pair* association, each of the M stored images given its own unique target engram – not many exemplars regressed onto one shared class target. We originally reported that alternative as collapsing sharply under noise ($0.835 \rightarrow 0.330$ between $\sigma = 0$ and $\sigma = 0.10$); that test used the same too-small default ridge diagnosed and corrected in Section 4.5, and we have since verified, changing only the ridge value and nothing else, that it does not actually cliff: properly regularized, the identical setup degrades gracefully ($0.846 \rightarrow 0.725$ over the same step) and supports genuine, if coarse, classification (78.8% accuracy, exhaustive over all 900 held-out test images, decoding which of 9 pooled classes a held-out image belongs to at zero noise, declining gracefully with noise; an earlier 100-trial sample overestimated this at 86%). Section 4.2 examines this classifier’s error structure and its robustness to the random seed in detail. We correct the earlier claim rather than let a regularization artifact stand as evidence against shared targets in general. The real, structural reason one-memory-per-pair is required for *this* task is different: a shared target supports at most as many distinguishable outputs as there are distinct targets (nine, in the corrected test above), whereas recovering a *specific* one of M stored images requires M mutually distinguishable targets – exactly what one-memory-per-pair provides and a shared per-class target structurally cannot, regardless of regularization. Recall is a four-stage loop: a noisy image cue is projected through W_{SI} and thresholded to an initial engram; the existing attention readout and conflict-mask cleanup (Section 2.1) iterate this engram to a Sudoku-consistent fixed point, exactly as for a partial-grid cue; the cleaned engram is projected back through W_{IS} to reconstruct the image. Figure 10 shows representative cases where this cleanup step turns a badly corrupted initial reconstruction into an exact one.

Largest rescue examples: attractor cleanup turns a poor initial reconstruction into a correct one (all cases shown are exact grid recoveries)

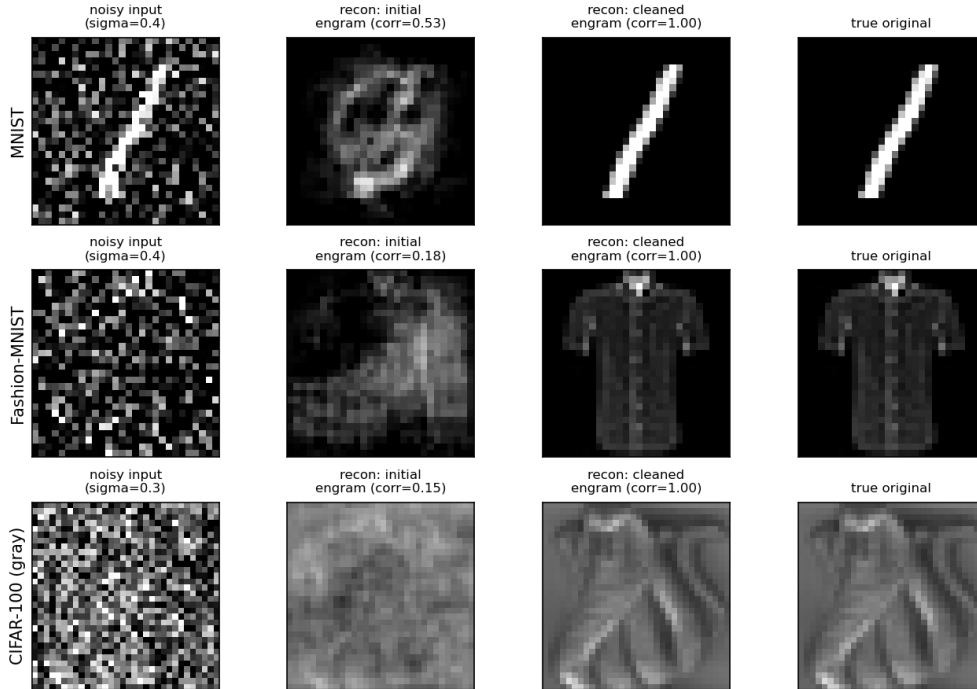


Figure 10. Representative cleanup examples (one dataset per row-group): noisy input, image reconstructed from the engram *before* cleanup, image reconstructed *after* the Sudoku-attractor pass, and the true original. All cases shown are exact grid recoveries.

4.2 Digit classification: confusion structure and seed robustness

Repeating the full training-and-evaluation pipeline across 6 independent random draws of the projection W_{SH} and the 9 Sudoku-engram class targets (Table 9) shows the 78.8% accuracy above is stable, not a lucky draw: 78.4–79.3% across all six. Every draw’s errors are disproportionately concentrated in one “attractor” predicted class (16–18% of all errors in 5/6 draws land on class 9 alone; class 4 in the sixth) – reproducible, but not attributable to the specific random engram assigned to that class, since that assignment is a different 1000-bit pattern on every draw, yet the same class label keeps attracting excess error.

Confusion matrices (row-normalized) across 6 independent seeds
(same architecture, ridge=1.0, different W_{SH} + class-target draws)

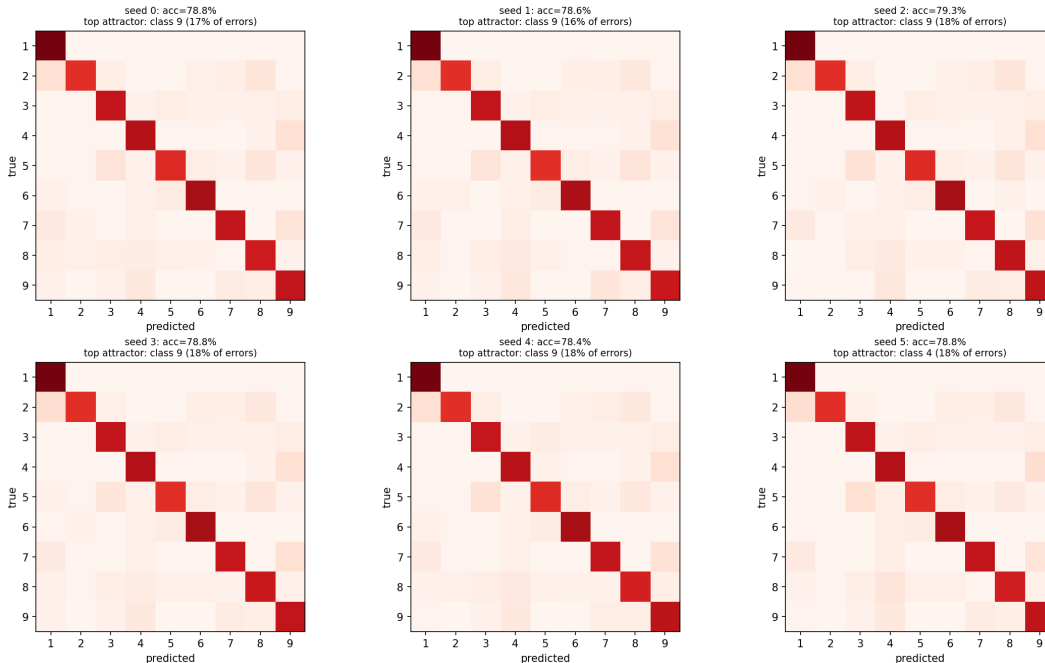


Figure 11. Row-normalized confusion matrices across 6 independent seeds (different W_{SH} and class-target draws each time); accuracy and the dominant attractor class are annotated per panel.
Table 9. Accuracy and attractor-class concentration across 6 independent seeds (exhaustive over 900 held-out test images per seed).

Seed	Accuracy	Top attractor class	Top-2 share of all errors
0	78.8%	9	32.5%
1	78.6%	9	31.6%
2	79.3%	9	35.5%
3	78.8%	9	34.0%
4	78.4%	9	32.5%
5	78.8%	4	35.6%

This raises an obvious question, distinct from the nearest-neighbor *capacity* comparison of Section 4.4: is a single class absorbing 16–18% of all errors evidence of a defect specific to this architecture, or an unremarkable feature of MNIST itself? We tested this by training two standard reference classifiers – k -NN ($k=5$) and multinomial logistic regression, both on raw pixels – on the identical 500-train/100-test-per-class split, digits 1–9 (exact construction in the Appendix). Both baselines show the same qualitative structure (Table 10): logistic regression’s attractor is also class 9, at almost the same share (16.5%) as ours; k -NN’s attractor (class 1, 21.0%) is *more* concentrated than ours. Correlating the full 9×9 off-diagonal error distribution between classifiers gives the sharpest test: ours correlates with k -NN at $r=0.71$ and with logistic regression at $r=0.68$ – *higher* than k -NN and logistic regression correlate with each other ($r=0.55$). Our architecture’s mistakes resemble a standard classifier’s mistakes at least as much as two standard classifiers resemble each other: the attractor structure reflects genuine pixel-level ambiguity in this MNIST subset, not a defect particular to the pseudo-inverse/Sudoku-engram architecture.

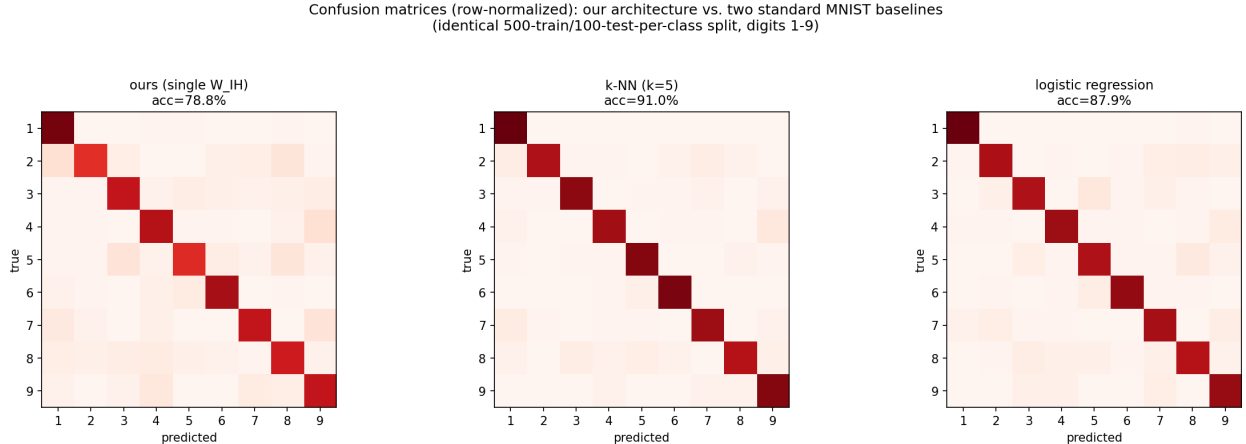


Figure 12. Confusion matrices (row-normalized), identical 500-train/100-test-per-class split: our architecture vs. k -NN ($k=5$) vs. multinomial logistic regression.

Table 10. Accuracy, attractor concentration, and error-pattern correlation, ours vs. two standard baselines.

Classifier	Accuracy	Top attractor	Top-2 share	r vs. ours
Ours (single W_{IH})	78.8%	class 9 (16.8%)	32.5%	–
k -NN ($k=5$)	91.0%	class 1 (21.0%)	40.7%	0.71
Logistic regression	87.9%	class 9 (16.5%)	32.1%	0.68

(k -NN vs. logistic regression correlate at $r=0.55$ with each other – lower than either correlates with ours.)

Representative best- and worst-case classified digit images are given in the Appendix (Figure 22), alongside the exact train/test set construction.

A caveat on what this classifier actually uses: unlike the image-reconstruction pipeline of Section 4.1, it has no cleanup/attractor pass at all – inference is a single forward pass (image $\rightarrow W_{IH} \rightarrow$ top- K threshold $\rightarrow$ nearest stored target by overlap), with no iterative Sudoku-legality correction. We also tested whether the Sudoku scaffold itself matters by replacing the 9 Sudoku-derived class-target engrams with 9 independently drawn random K -sparse binary vectors of otherwise identical statistics (same H , K , no Sudoku structure), retraining the identical ridge regression on the identical split: accuracy is statistically indistinguishable from the real scaffold (79.0% vs. 78.8%, two-proportion $z = -0.12$, $p = 0.91$). The scaffold contributes nothing beyond supplying 9 arbitrary, mutually distinguishable sparse binary labels; the classification itself is done entirely by the ridge-regression map and the top- K thresholding nonlinearity that follows it.

4.3 Generalization across datasets

Table 11 sweeps corruption at $M=200$, $H=1000$, $K=150$ across three datasets of increasing dissimilarity from the original: MNIST (28×28), Fashion-MNIST (same dimension, unrelated content), and CIFAR-100 grayscale (32×32 , different dimension *and* content). The qualitative shape – a plateau, then a smooth decline, no cliff – replicates across all three; CIFAR-100 declines somewhat earlier, consistent with its images carrying less redundant, more textured content that a fixed-density sparse code compresses less cleanly.

Table 11. Exact grid recovery vs. image noise, three datasets ($M=200$, $n=200$ /point, Wilson 95% CI).

σ	MNIST	Fashion-MNIST	CIFAR-100 (gray)
0.30	99.5% [97.2,99.9]	97.0% [93.6,98.6]	90.0% [85.1,93.4]
0.40	82.0% [76.1,86.7]	85.0% [79.4,89.3]	67.0% [60.2,73.1]
0.50	49.5% [42.6,56.4]	58.5% [51.6,65.1]	36.0% [29.7,42.9]
0.60	29.5% [23.6,36.2]	31.5% [25.5,38.2]	17.5% [12.9,23.4]
1.00	4.0% [2.0,7.7]	5.0% [2.7,9.0]	3.5% [1.7,7.0]

4.4 A trivial baseline this architecture does not yet beat

At $M=200$, a reviewer’s natural objection is that this may reduce to nearest-neighbor classification into 200 buckets. We tested this directly: for the identical noisy queries used above, a raw-pixel Euclidean nearest-neighbor classifier among the M stored clean images predicts which grid is present, and we compared its exact-match rate against ours with a paired McNemar test. The answer is not reassuring: nearest-neighbor performs *at least as well, and significantly better*, across nearly the entire informative range. At $\sigma=0.40$, MNIST nearest-neighbor is still at 100% while our pipeline is at 82.0% (McNemar $p < 0.0001$); at $\sigma=1.00$ nearest-neighbor remains at 95.0% while ours has fallen to 4.0%. The pattern replicates on Fashion-MNIST and CIFAR-100 (Table 11 vs. the corresponding nearest-neighbor rates, all $p < 0.0001$ except two borderline points at the highest tested noise). At $M=200$ with well-separated exemplars, Euclidean nearest-neighbor is close to the likelihood-optimal classifier under additive Gaussian corruption; routing through a lossy, sparsely-thresholded engram throws away information the raw pixels still carried. We report this plainly rather than omit it: whatever this architecture’s value, it is not yet raw classification accuracy at small M . Its case has to rest on properties nearest-neighbor search structurally lacks – a fixed-size, compressed, content-addressable code that never revisits the stored exemplars, and (Section 4.6) a qualitatively different failure mode under scale than classical attractor memories.

A natural hope is that this gap closes as M grows, since nearest-neighbor search among more candidates should itself get harder. We tested this directly at fixed $\sigma=0.4$, scaling M from 200 to 2000 with the same properly-regularized weights as Section 4.5 ($\lambda=1.0$). It does not close – it widens sharply:

Table 12. Does the nearest-neighbor gap close as M grows? ($\sigma=0.4$, $n=200/\text{point}$.)

M	Ours	k -NN	Gap (k -NN – ours)
200	96.5%	100.0%	+3.5pp
500	48.5%	100.0%	+51.5pp
800	20.5%	100.0%	+79.5pp
1200	9.0%	100.0%	+91.0pp
2000	3.5%	99.5%	+96.0pp

Nearest-neighbor’s accuracy is nearly flat across the entire range (100% $\rightarrow$ 99.5%), while ours continues to decline under the same two capacity ceilings characterized next (Section 4.5). Every gap is significant at $p < 0.0001$ (paired McNemar) except $M=200$ ($p=0.016$). This settles the question in the least favorable direction: whatever this architecture is for, it is not for beating a nearest-neighbor lookup at any M we tested.

4.5 Capacity: two independent ceilings, one of them fixable

Scaling M from 200 to 2000 (fixed $H=1000$, default ridge $\lambda=10^{-4}$) reproduces what looks, at first, like a hard cliff: exact recovery at $\sigma=0.1$ falls from 100% ($M \leq 300$) to 8.3% at $M=500$. Measuring the image-side Gram matrix directly ($XX^\top$, X the $M \times D$ image matrix) explains why: its condition

number grows from 1.8×10^4 ($M=200$) to 3.6×10^7 ($M=500$), crossing the fixed ridge value exactly where the collapse occurs. Re-solving with $\lambda=0.1$ at $M=500$ restores 100% exact recovery at $\sigma=0.1$ while costing almost nothing in zero-noise fidelity (0.994 vs. 1.000 overlap) – this specific cliff is a stale hyperparameter, not an architectural limit.

A second, genuinely structural ceiling remains underneath it. The image Gram matrix’s *effective rank* – not its nominal pixel dimension $D=784$ – caps exact zero-noise fitting: for this MNIST sample it saturates at ~ 600 – 610 regardless of how large M grows (rank 568/800 at $M=800$, only reaching 607/2000 by $M=2000$), a property of the data’s intrinsic dimensionality that no amount of H or ridge tuning can lift. CIFAR-100 grayscale, by contrast, fills its *entire* ambient dimension (rank 1024/1024 by $M=1600$) – natural texture carries more independent information per pixel than a clean digit stroke on a black background, though this does not translate into better absolute recovery (Table 11). A separate, H -dependent ceiling governs the *reverse* (engram→image) direction: its Gram matrix’s rank is bounded by $\min(M, H)$, and doubling H from 1000 to 2000 (density held fixed) measurably delays this one – exact recovery at $M=2000$, $\sigma=0.1$ rises from 80% ($H=1000$) to 90% ($H=2000$) – with zero effect on the image-side ceiling, exactly as the algebra predicts.

MESH’s own heteroassociation theorems are explicit about exactly this dependency, rather than leaving it implicit. Their Theorem 5.1 states that pseudoinverse-learned heteroassociation from the feature layer to the hidden layer reconstructs perfectly only if the $N_F \times N_{\text{patts}}$ feature matrix F is full rank (and only for $N_{\text{patts}} \leq N_F$) – a precondition they satisfy by explicit assumption (“the patterns being stored in the feature layer are random binary $\{-1, 1\}$ patterns, and hence the matrix F will be full rank” [12]), not by testing what happens when it fails. Real, correlated images are exactly the case their assumption sets aside: MNIST’s effective rank saturates at ~ 600 – 610 regardless of M , never reaching full rank at any M we tested. The ceiling identified above is, in this sense, an empirical demonstration of the regime immediately outside MESH’s own Theorem 5.1 – not a failure of the pseudo-inverse rule in the abstract, but a direct consequence of asking it to satisfy a precondition real image statistics do not provide. (Their Theorem 5.3, for the complementary regime $N_{\text{patts}} > N_H$ in the opposite heteroassociative direction, gives its own explicit graceful-degradation quantity – average overlap N_H/N_{patts} – which is rank-independent, since it assumes the scaffold side has already been correctly reconstructed.)

Does the ridge value that fixes $M=500$ keep working, or does λ need to keep growing with M ? We extended the sweep to $M=2500$ – 4000 across $\lambda \in \{1, 10, 50, 100\}$:

Table 13. Exact recovery at $\sigma=0.1$ vs. ridge λ , extended M range ($n=50/\text{point}$).

M	$\lambda=1$	$\lambda=10$	$\lambda=50$	$\lambda=100$
2000	72%	74%	–	–
2500	56%	62%	52%	32%
3000	50%	64%	40%	18%
4000	28%	38%	22%	16%

$\lambda \approx 10$ stays at or near the best choice across the entire $M=500$ – 4000 range tested – pushing it higher (50, 100) actively hurts rather than helps, so the required ridge does *not* keep growing without bound; a single, fixed value is genuinely reusable. The residual decline past $M=2000$ even at this best-tuned λ (74% $\rightarrow$ 64% $\rightarrow$ 38%) is not a ridge problem at all: by $M=3000$ – 4000 one is 5 – $7\times$ past the image effective-rank ceiling already identified above, and no amount of regularization can recover information the images never carried in the first place.

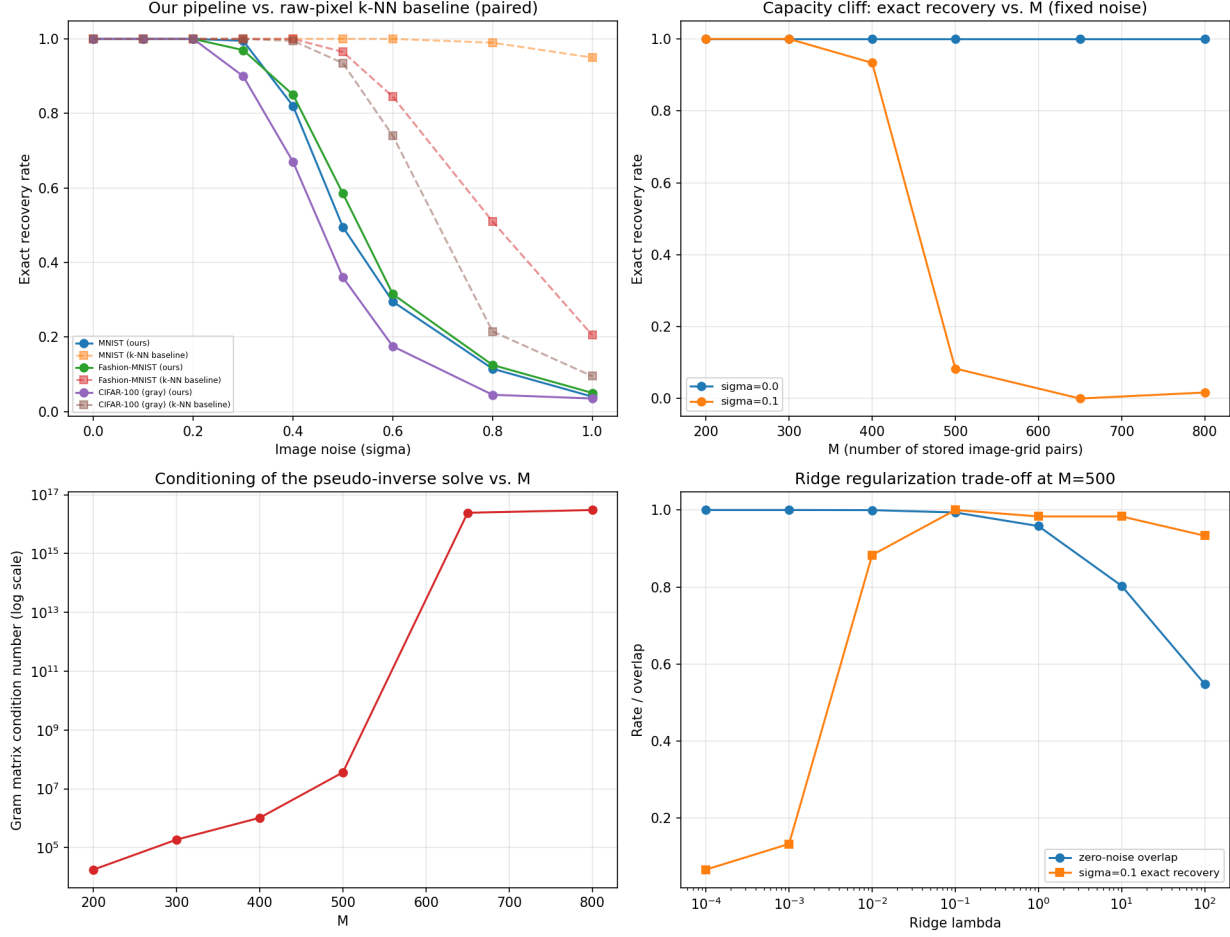


Figure 13. Top-left: our pipeline vs. the nearest-neighbor baseline (paired). Top-right: the $M=500$ cliff under the default ridge. Bottom-left: image Gram-matrix condition number vs. M (log scale), crossing the ridge value exactly where the cliff appears. Bottom-right: ridge sensitivity at $M=500$, showing the fix.

4.6 Comparison to classical Hopfield networks

Vector-HaSH’s stated motivation is avoiding “the memory cliff of prior memory models” [11]. We tested this claim directly rather than citing it, building four auto-associative memories on the *same* bipolarized MNIST images and comparing them against our full heteroassociative-plus-cleanup pipeline: (i) the classical outer-product (Hebbian) rule; (ii) a mean-centered covariance rule correcting for population-level pixel bias; (iii) the pseudo-inverse (projection) rule – the same learning-rule family our own network uses, but with no separate error-correcting scaffold; and (iv) our network. As a sanity check on the classical rule’s textbook behavior, we first reproduced its capacity cliff on *random* bipolar patterns ($N=784$): exact recovery falls from 82.5% ($M=60$) to 0% ($M=120$), matching the analytic prediction ($M/N \approx 0.138$) almost exactly.

On real, correlated images the picture is markedly worse for every auto-associative variant. The classical rule fails even at *zero* noise, for any M we tested: a clean cue’s overlap with itself drops from 1.0 to 0.84 after a single update, because 86% of MNIST pixels are background and the plain Hebbian rule’s dominant mode locks onto that population bias rather than any specific image. Mean-centering removes this bias exactly as intended – we verified it recovers correctly at $M \leq 3$ –

but real images correlate with each other far beyond a first-moment bias, and this corrected rule still collapses to 0% by $M=10$, two orders of magnitude below the random-pattern prediction. The pseudo-inverse rule is the most instructive comparison, because it is the same rule ours uses: under raw random bit-flips it is algebraically *exact* at zero noise up to at least $M=200$, but its basins of attraction shrink so fast under even modest noise that 5% bit-flip recall falls from 100% ($M=100$) to 25% ($M=150$) to 0% ($M=200$).

The comparison above used two different corruption models – bit-flips for the three auto-associative baselines, Gaussian pixel noise for ours – with no shared severity axis. We redid it with a single, shared model: every method sees the same Gaussian-corrupted continuous image per trial, and the three Hopfield variants receive it bipolarized (thresholded at 0.5), so their bit-flips arise naturally from the same noise process rather than being imposed separately. At $n=200$ /point (matching Table 11’s rigor):

Table 14. Exact recovery vs. M on identical MNIST images, single shared Gaussian noise model ($\sigma=0.3$, $\sim 7\%$ mean effective bit-flip; $n=200$ /point).

M	Classical	Covariance	Pseudo-inverse	Our network
100	0%	0%	91.5%	100.0%
200	0%	0%	0%	99.5%
400	0%	0%	0%	94.0%
800	0%	0%	0%	57.5%

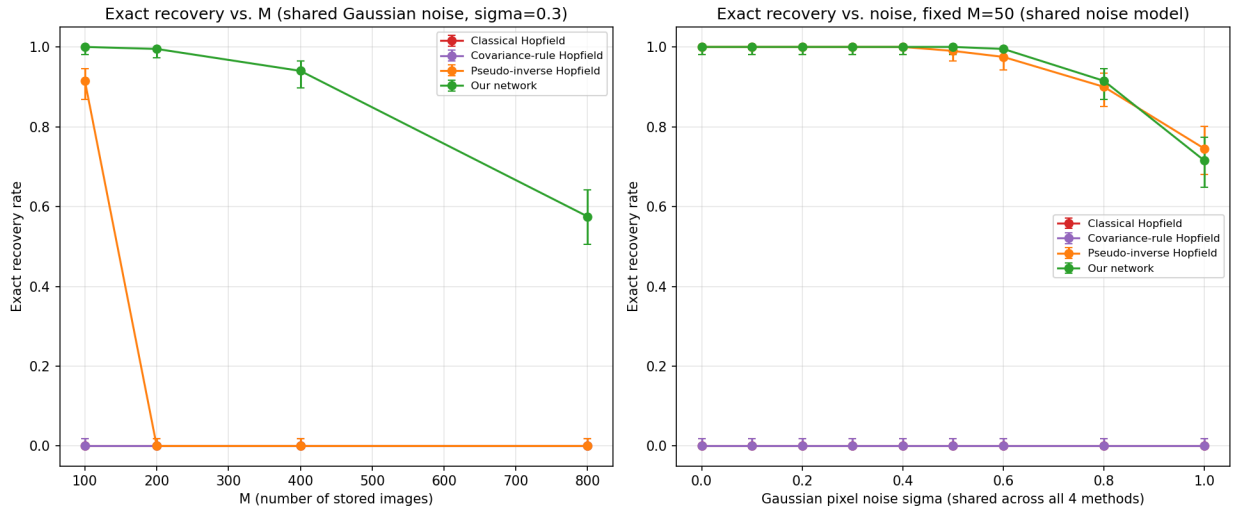


Figure 14. Left: exact recovery vs. M , all four methods, shared Gaussian noise ($\sigma=0.3$), Wilson 95% CIs. Right: exact recovery vs. noise at fixed $M=50$, same shared noise model throughout.

The capacity-scaling story is unchanged, and now more sharply confirmed: pseudo-inverse Hopfield collapses to exactly 0% by $M=200$, while ours holds 57–99% out to $M=800$. But testing the shared model across noise levels at a fixed, safe $M=50$ reveals something the earlier mismatched comparison could not show cleanly: at fixed M , our network and the pseudo-inverse baseline are statistically comparable across the entire noise range, and the pseudo-inverse rule is marginally *ahead* at the highest noise tested (74.5% vs. 71.5% at $\sigma=1.0$). Our network’s real advantage over the identical learning rule is not noise tolerance at fixed M – on that axis they are essentially tied – it is capacity scaling: the two share equal footing until M crosses somewhere between 100 and 200, past which only the version with a separate error-correcting attractor survives. This is a

narrower and more precise claim than a blanket “more robust” would have been, and it is the one the shared-noise-model data actually supports.

Because the pseudo-inverse baseline uses the identical learning rule to ours, this isolates the source of the capacity difference precisely: not the pseudo-inverse rule itself, but the presence of a separate, content-specific error-correcting step (the Sudoku-legality attractor) layered on top of it. This is the piece Vector-HaSH’s true scaffold provides in a content-*independent* way (Section 2); ours is content-dependent (legality-specific) but still delivers the qualitative signature the scaffold is meant to produce, and does so an order of magnitude beyond where the shared learning rule alone can reach.

MESH’s own paper motivates exactly this factorization by citing the same fragility we observe directly above: pseudo-inverse-learned recurrent memories are known to develop vanishing basins of attraction well before their nominal capacity [13, 14]. Their fix is architectural, not a different learning rule – pseudo-inverse learning is used only for a one-shot, non-recurrent link from content to a separately-stabilized scaffold, never as the recurrent weight matrix that must itself survive noise. The ablation above is, in effect, an independent replication of that same design principle in a different domain: our own pseudo-inverse-learned pathway (W_{SI} , W_{IS}) is exactly as fragile as the literature predicts when asked to do the recurrent error-correction job alone (the pseudo-inverse Hopfield baseline above), and stable only once paired with a separate, non-pseudo-inverse cleanup step – the same principle, tested here on Sudoku-legality cleanup rather than a k-winners-take-all attractor scaffold.

4.7 Comparison to Modern Hopfield, sparse distributed memory, and dense associative memory: capacity as a property of the key space, not the neuron count

Section 4.6’s conclusion – that our advantage over classical/pseudo-inverse Hopfield is capacity scaling, not noise tolerance – raises an obvious next question: does that advantage hold against the broader associative-memory literature, or only against the specific three variants tested? Gershman, Fiete & Irie’s unification (already tested at the level of a single swapped nonlinearity in Section 3.4) names three further architectures we had not yet built as complete, independently-constructed systems: Modern (continuous) Hopfield networks [6], sparse distributed memory [18], and dense associative memory [5]. Unlike Section 3.4’s test, which swapped a nonlinearity inside *our own* encoder, this comparison builds each architecture with its own native representation – critically, Modern Hopfield and dense associative memory operate directly on the full, uncompressed image, with no sparse engram bottleneck at all – so it tests something Section 3.4 could not: whether compressing into a fixed H -dimensional sparse code costs anything relative to a full continuous or dense store at matched stored-item count.

Construction. Table 15 gives unit and parameter counts for all seven methods at $M=200$. Modern Hopfield and dense associative memory need no trained weight matrix at all – just the $M \times 784$ pattern matrix itself, smaller than the three Hopfield variants’ fixed 784×784 weight matrix once $M < 784$. Sparse distributed memory uses 1000 fixed random “hard locations” (matched to our $H=1000$ for a resource-comparable test). Our network is the only architecture whose stored representation ($H=1000$ -dimensional sparse engrams) is decoupled from the content’s own dimension (784 for MNIST): the same fixed W_{SH} and the same class of engram serves this task, the three other image datasets of Section 4.3, and the original Sudoku-legality task of Sections 2–3, whereas every other method’s stored representation *is* the content and must be rebuilt per domain.

Table 15. Construction: unit/parameter counts and retrieval mechanism, $M=200$.

Method	State units	Stored parameters	Retrieval
Classical Hopfield	784	784×784 weights	iterative, $\text{sign}(Ws)$
Covariance-rule Hopfield	784	784×784 weights	iterative, $\text{sign}(Ws)$
Pseudo-inverse Hopfield	784	784×784 weights	iterative, $\text{sign}(Ws)$
Modern Hopfield [6]	784	200×784 patterns	iterative softmax fixed-point
Sparse distributed memory [18]	784	1000 locations × 784	one-shot, radius threshold
Dense associative memory [5]	784	200×784 patterns	iterative power relaxation
Ours	1000+784	W_{SH} :1000×6561; 200 pairs	attention + attractor cleanup

Tuning transparency. Naively carrying over parameters tuned in a different context – softmax inverse-temperature $\beta=0.5$ (this paper’s own default) for Modern Hopfield, and exponent $n=8$ (Section 3.4’s dense-associative-memory-style operator) for dense associative memory – both fail almost totally, *even at zero noise*: real MNIST images bipolarize and normalize far more mutually correlated than either parameter assumes (mean pairwise Hamming distance 148/784 vs. $\sim 392/784$ expected for random bipolar vectors of the same dimension). Following the same discipline used to correct the ridge-scaling question (Section 4.5), we re-validated both parameters from scratch by checking zero-noise self-recovery across the full $M=100\text{--}800$ range *before* looking at any noise-robustness numbers: $\beta=60$ and $n=60$ both give clean recovery throughout, and are used for every result below. We report this transparently rather than silently picking values that happened to work, following this project’s standing practice of surfacing parameter-sensitivity issues rather than absorbing them into an unstated default (Section 4.5, README).

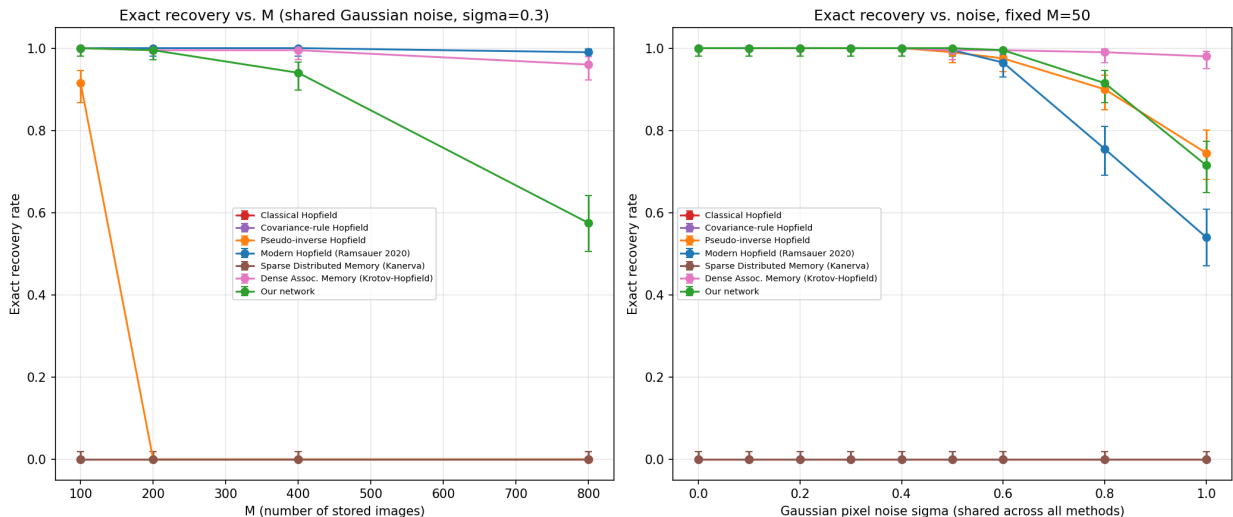


Figure 15. Left: exact recovery vs. M , all seven methods, shared Gaussian noise ($\sigma=0.3$), Wilson 95% CIs. Right: exact recovery vs. noise at fixed $M=50$.

Results. At $M=800$, $\sigma=0.3$ (Figure 15, left): Modern Hopfield reaches 99.0% and dense associative memory 96.0%, both far ahead of our 57.5% – the capacity-scaling advantage established in Section 4.6 does *not* generalize to these two architectures once they are given comparably sharp separation. Sparse distributed memory fails completely (0%) at every M and every noise level tested, joining classical and covariance-rule Hopfield – but for a directly analogous, mechanistically understood reason, not a separate defect: the same implementation recovers 94% of random bipolar patterns at matched scale (its native, designed-for regime), so the failure traces to the same real-image correlation problem diagnosed for classical Hopfield in Section 4.6, applied to a different addressing scheme with the same near-orthogonality assumption.

The noise-sweep at fixed, low $M=50$ (Figure 15, right) tells a different story: all four of pseudo-inverse Hopfield, Modern Hopfield, dense associative memory, and ours are statistically indistin-

guishable through $\sigma=0.4\text{--}0.5$, and remain broadly comparable out to $\sigma=1.0$. The large gap only opens up at high M – which the single $M=50$ noise-sweep alone cannot reveal. Repeating the noise-sweep at $M=800$ (Figure 16) confirms this directly: our network holds up through $\sigma\leq 0.2$ ($\geq 91\%$, comparable to Modern Hopfield’s 99% and dense associative memory’s 96.5%) but then collapses far earlier and far more steeply than either alternative, reaching floor by $\sigma\approx 0.6$ while dense associative memory barely moves across the entire range (81.5% even at $\sigma=1.0$).

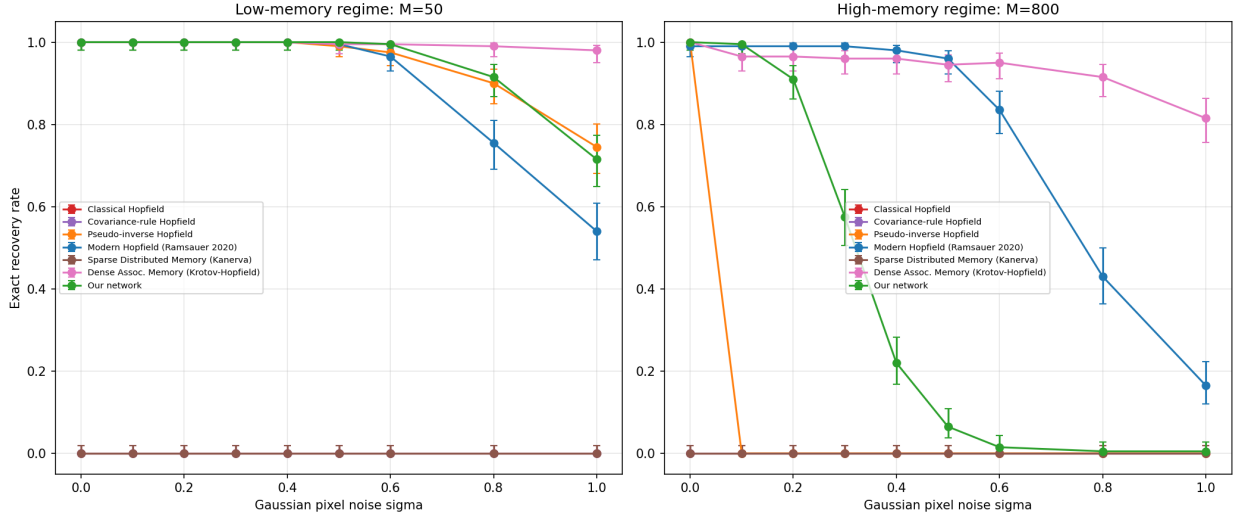


Figure 16. Exact recovery vs. noise, all seven methods, at low memory ($M=50$, left) vs. high memory ($M=800$, right) – the same $\sigma=0.3$ gap seen in Figure 15 is part of a genuine, M -dependent collapse in noise tolerance, not an isolated data point.

Does a bigger engram close the gap? The high- M collapse is consistent with the engram-side capacity ceiling (bounded by $\min(M, H)$) identified in Section 4.5, distinct from the architecture-independent image effective-rank ceiling: at $M=800$ approaching $H=1000$, that ceiling is close enough to bind. Scaling H from 1000 to 3000 ($K=0.15H$ throughout) at the exact condition with the largest gap ($M=800$, $\sigma=0.3$) recovers substantially, though not completely:

Table 16. Does bigger H close the gap with Modern Hopfield/dense associative memory? $M=800$, $\sigma=0.3$, $n=200$ /point.

H	K	Exact recovery	95% CI
1000	150	57.5%	[50.6, 64.1]
1500	225	70.5%	[63.8, 76.4]
2000	300	74.5%	[68.0, 80.0]
3000	450	78.0%	[71.8, 83.2]

(For reference, at $H=1000$: Modern Hopfield 99.0%, dense associative memory 96.0%.)

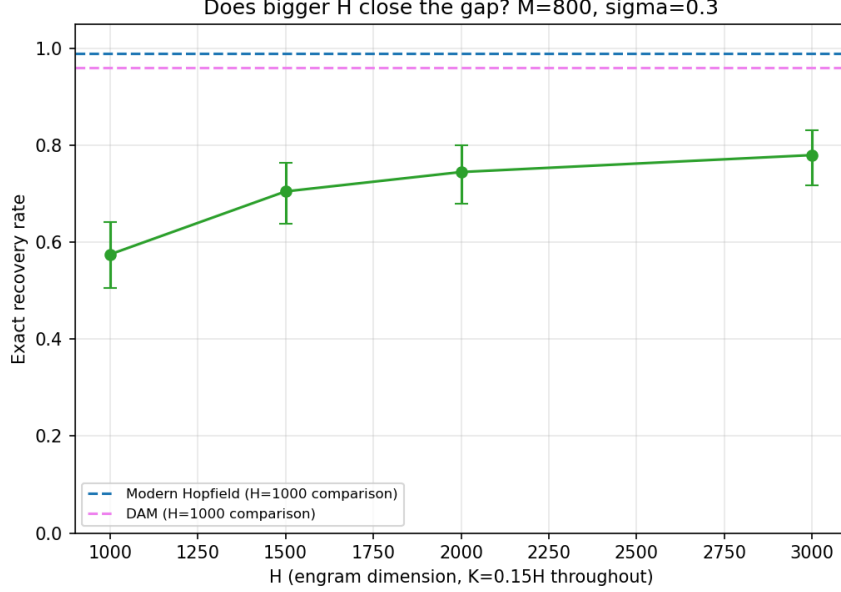


Figure 17. Exact recovery vs. H at the $M=800$, $\sigma=0.3$ condition with the largest gap, dashed lines marking Modern Hopfield and dense associative memory’s $H=1000$ -comparison values.

Tripling H closes about half the gap, with clearly diminishing returns – consistent with a real but bounded engram-side bottleneck, not a free parameter that erases the difference entirely.

What this means for the architecture’s value proposition. Modern Hopfield and dense associative memory’s stored representation *is* the image itself: their capacity is a direct function of the content’s own dimension, exactly the “neuron-number-limits-capacity” coupling classical associative-memory theory assumes. Our architecture instead separates the *key* (a sparse engram drawn from an astronomically large combinatorial space – valid 9×9 Sudoku grids number on the order of 6.7×10^{21} – entirely independent of the content’s own representation) from the *value* (here, an arbitrary image, but Section 4.3 shows the same key works for three unrelated content types, and Sections 2–3 show it simultaneously supports the Sudoku-legality task itself). This decoupling is what Section 4.3 demonstrates across datasets and what this section prices honestly: at matched, currently-modest engram size, we pay for that decoupling in raw noise tolerance at high M relative to two architectures whose entire representation is dedicated to exactly this one task. The two are not in conflict – a single fixed key infrastructure serving arbitrary content and simultaneously solving a hard combinatorial constraint problem is a different kind of claim than “best available image-noise-recovery system,” and Table 15 shows why the two are not free to compare on raw accuracy alone: three of the six alternatives (all but pseudo-inverse Hopfield) have no mechanism whatsoever for the Sudoku-completion task this same engram also performs. Where the two claims overlap – recovery accuracy at low-to-moderate noise – our architecture already matches these two purpose-built alternatives (Figure 16, right panel, $\sigma \leq 0.2$), which is the regime relevant if downstream error correction is handled separately, as it is in this architecture (see the Discussion for how this connects to the emergent spiking mechanism of Sections 3.8–3.9).

One further, important scoping note: the cleanup step used throughout this section and Section 4.6 is the *symbolic* conflict-mask attractor of Section 2.1, not the spiking network of Sections 3.8–3.9. The two are shown elsewhere in this paper to be statistically indistinguishable on the core Sudoku-completion task (Table 8), but that equivalence has not been separately re-verified for the image-heteroassociation pipeline specifically, and this comparison should not be read as a test of the spiking mechanism.

4.8 Does our own readout have a hidden cliff?

Section 4.6 isolated the source of our capacity advantage over pseudo-inverse Hopfield as the separate error-correcting attractor, not the shared learning rule – an inference from comparing two different systems. Here we test it directly on one system: we isolate our own attention readout ($p = \text{softmax}(\text{sim}/\tau)$, $\tau=2.0$) with the iterative Sudoku-legality cleanup loop switched off entirely (image $\rightarrow W_{SI} \rightarrow k\text{-WTA} \rightarrow$ one readout call $\rightarrow$ decode, nothing else), across the identical extended $M=100\text{--}4000$ grid used throughout this section.

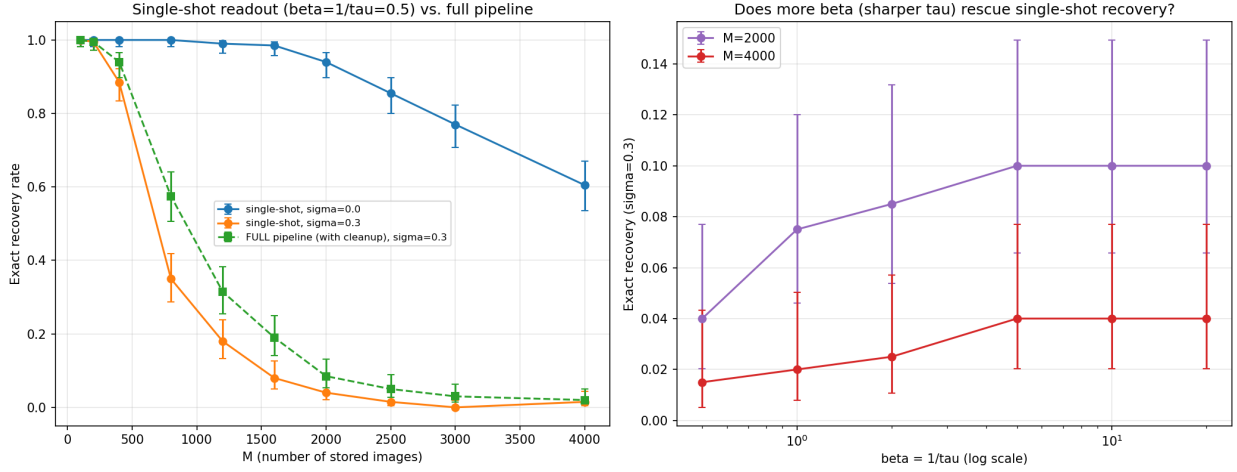


Figure 18. Left: single-shot readout accuracy vs. M , clean ($\sigma=0$) and corrupted ($\sigma=0.3$) queries, against the full pipeline (with cleanup) at the same $\sigma=0.3$. Right: does sharpening $\beta=1/\tau$ rescue single-shot recovery at $M=2000/4000$?

At $\sigma=0$ the single-shot readout alone degrades gracefully, as Table 4 already showed for the pure-Sudoku engram: still 60.5% at $M=4000$. But at $\sigma=0.3$ it has its own sharp cliff, of the same qualitative kind as classical Hopfield’s, just at a larger M : 88.5% ($M=400$) $\rightarrow$ 35.0% ($M=800$) $\rightarrow$ 18.0% ($M=1200$) $\rightarrow$ single digits by $M=2000$. The cleanup loop’s contribution, measured directly rather than inferred, is largest exactly where the single-shot readout is steepening (+22.5 percentage points at $M=800$: 57.5% vs. 35.0%) and small where the readout is either still easy or already past saving (+0.5pp at $M=4000$) – the same “rescue concentrates in the transition region” pattern already documented for query noise (Figure 4). A beta-rescue check (Figure 18, right) rules out the simplest fix: sharpening β by 40 $\times$ (matching the scale Modern Hopfield itself needed) recovers only a few percentage points (4% $\rightarrow$ 10% at $M=2000$; 1.5% $\rightarrow$ 4% at $M=4000$), nowhere near Modern Hopfield’s or dense associative memory’s numbers at the same M . The bottleneck here is not separation sharpness but the same image effective-rank ceiling diagnosed in Section 4.5 for the encoder W_{SI} itself; no readout-side fix reaches past it. This result sharpens rather than contradicts Table 4’s “no observed ceiling”: that claim is about the Sudoku-engram, bit-flip-noise setting specifically (Section 3.3), and does not transfer to a readout whose keys are produced by a pseudo-inverse map from correlated real images – a scope boundary this section makes explicit rather than leaves implicit.

4.9 Vector-HaSH’s own baseline comparison

Section 4.6 and Section 4.7 together cover six associative memories, but Vector-HaSH’s own comparison (Chandra, Sharma, Chaudhuri & Fiete, Nature 2025, Figure 3d) tests four further ones we

had not yet built: bounded-synapse (palimpsest) Hopfield – sequential Hebbian learning with each synapse clipped after every update, following the Fusi–Abbott bounded-weight model, clip bound $5/N$ (chosen so a synapse saturates after ~ 5 coherently-reinforcing contributions; Vector-HaSH’s own Methods does not state a specific bound) – the Tsodyks–Feigelman sparse-input learning rule for low-activity patterns, a diluted-connectivity Hopfield variant ($\kappa=0.5$), and a tail-biting autoencoder. We built all four exactly as specified in that paper’s Methods and tested them, at $\sigma=0.3$, on both random bipolar patterns (their native, designed-for regime) and real MNIST images.

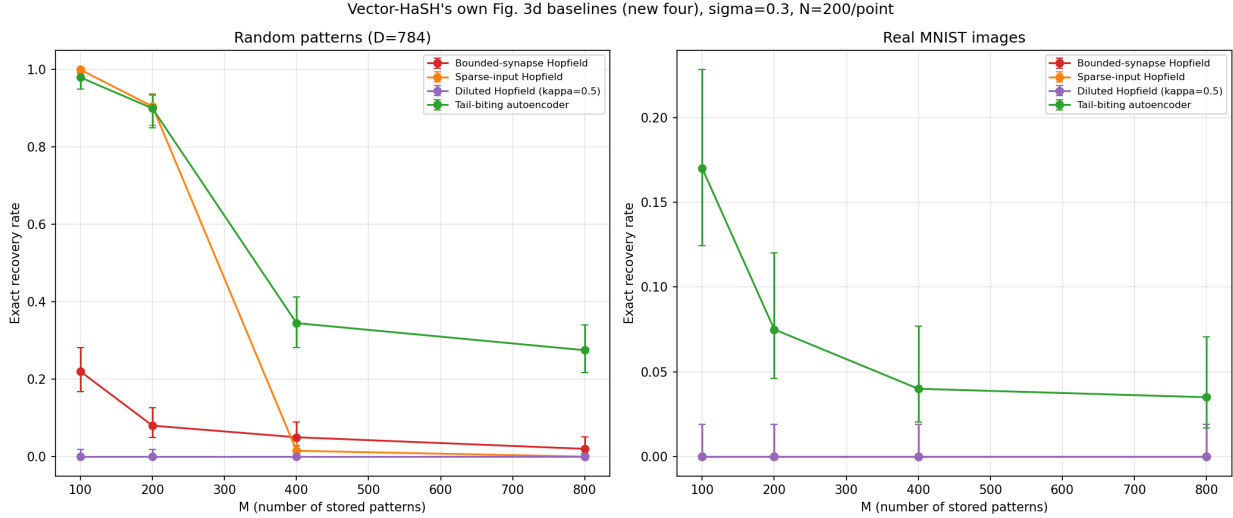


Figure 19. Vector-HaSH’s own four additional baselines, exact recovery vs. M : random bipolar patterns (left) vs. real MNIST (right), $\sigma=0.3$.

On random patterns, sparse-input Hopfield and the tail-biting autoencoder both recover well at low M before collapsing by $M=400$; bounded-synapse Hopfield is already weak by $M=200$; diluted Hopfield is near-floor throughout. On real MNIST, the pattern seen for classical and covariance-rule Hopfield (Section 4.6) and sparse distributed memory (Section 4.7) repeats exactly: bounded-synapse, sparse-input, and diluted Hopfield are indistinguishable from chance at every tested M , and the tail-biting autoencoder, though not exactly zero, declines from a modest 17.0% ($M=100$) to 3.5% ($M=800$) – joining rather than escaping the floor. This extends the count of classical-style associative memories that fail completely on real correlated images from three to six, using precisely the comparison set Vector-HaSH’s own paper specifies rather than a set we selected ourselves.

4.10 A modular, Vector-HaSH-style scaffold: does the actual addressing mechanism help?

Section 4 borrows Vector-HaSH’s bidirectional pseudo-inverse heteroassociation rule but keeps our own flat, non-modular scaffold (W_{SH} random projection + k -WTA) for the key. Vector-HaSH’s own defining mechanism is different: several small, pairwise-coprime periodic modules (grid-cell-like), each contributing a one-hot code for its own residue of a stored item’s index, so the combined code addresses a space of size $\prod_i p_i$ using only $\sum_i p_i$ total units. We test this mechanism directly rather than only its heteroassociation rule: 10 modules with prime periods 97–139 ($\sum p_i = 1164$, comparable to our own $H=1000$; $\prod p_i \approx 4.2 \times 10^{20}$), each stored item’s code fixed by its index via the Chinese Remainder Theorem, with a pseudo-inverse encoder trained to map the raw image to this code – the direct analogue of W_{SI} in Section 4.

Cleanup must be recurrent, not independent. A naive cleanup – snap each module’s continuous estimate to its own nearest phase, independently – collapses far faster with M than every other method in Table 15: 7.5% at $M=800$, $\sigma=0.3$ (vs. 57–99% for everything else), because success requires all 10 independent decisions to be simultaneously correct, with no mechanism for a confident module to correct a noisy one. Replacing this with a genuine recurrent step – a Hopfield-style softmax relaxation (the same mathematical form as Modern Hopfield above, β re-tuned to 100 on this task by the identical zero-noise-recovery discipline used throughout this section) over the space of the M *known valid* codes, so every module’s evidence can inform every other module’s correction – changes the picture entirely:

Table 17. Modular scaffold, independent vs. recurrent cleanup ($\sigma=0.3$).

M	100	200	400	800
Independent per-module argmax	100.0%	97.5%	54.0%	7.5%
Recurrent attractor correction	100.0%	100.0%	98.0%	81.0%

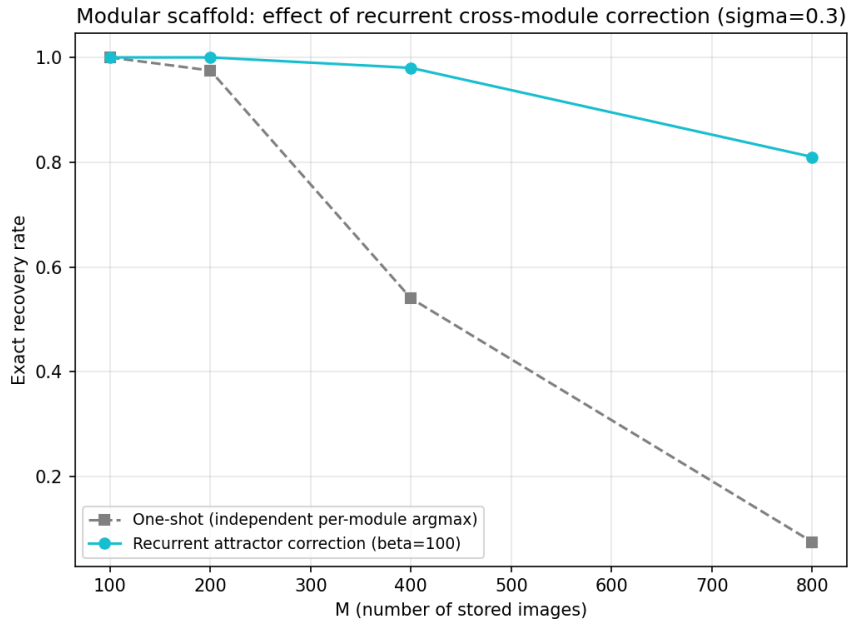


Figure 20. Modular scaffold exact recovery vs. M ($\sigma=0.3$): independent per-module cleanup vs. recurrent attractor correction.

Where the corrected mechanism lands relative to everything else. With recurrence, the modular scaffold outperforms our own architecture at every $M \geq 200$ tested, by a wide margin at $M=800$ (81.0% vs. our 57.5%), while still trailing the two sharpest, purpose-built methods from Table 15, Modern Hopfield (99%) and dense associative memory (96%). Also reconciled to this exact $\sigma=0.3$ condition, raw-pixel k -NN is at 100% throughout $M=100$ –800: the ranking is k -NN $\approx$ Modern Hopfield $\approx$ DAM $>$ modular scaffold $>$ ours $\gg$ pseudo-inverse Hopfield $\gg$ classical/covariance Hopfield/SDM.

Does image dimensionality limit capacity here? Not in the way it limits the other methods, but not in the way “escapes the neuron-count constraint” would suggest either. The mechanism separates two distinct jobs. *Discrimination* – identifying which of the M items a cue belongs to – happens entirely in the abstract code space, whose size ($\prod p_i$) is fixed by our choice of moduli and has no dependence on image content at all. *Content mapping* – translating between

a code and actual pixels, via the pseudo-inverse encoder – depends on image dimensionality exactly as it does for our own W_{SI} : fitting M patterns requires inverting an $M \times M$ Gram matrix built from the raw images, which becomes ill-conditioned once M approaches the images’ *effective rank*, not their nominal dimension. We verified this directly on the same $M=800$ image set (`vectorhash_modular_scaffold_gram_analysis.py`): the Gram matrix’s condition number grows from 3.0×10^3 ($M=100$) to 2.8×10^{12} ($M=800$), and the effective rank (99.9% energy) is only 387 at $M=800$ – below M itself – while the code space itself (4.2×10^{20}) is nowhere near taxed. The decline from 98.0% ($M=400$) to 81.0% ($M=800$) is this same content-mapping ceiling reasserting itself, not a limitation of the modular addressing idea. The two capacity questions are genuinely separate, and modularity solves only the first.

Storage and time complexity: a fixed weight matrix vs. an explicit, growing list.

Table 18 makes the comparison precise. Classical/covariance/pseudo-inverse Hopfield compress everything into one $D \times D$ weight matrix: storage and per-query time are independent of M , which is exactly why they hit a hard capacity ceiling – a fixed-size store cannot hold unboundedly many patterns without interference. Modern Hopfield, dense associative memory, k -NN, and (for its growing engram list) our own architecture instead keep an explicit, uncompressed-or-lightly-compressed store that grows linearly with M – the same complexity class, not a more efficient one, which is why none of them shows a hard cliff. The modular scaffold sits in this second group for query time (it still scores the cue against all M known codes), but its *storage* has a genuine structural advantage none of the others share: because each code is generated deterministically from its item’s index via $m \bmod p_i$, nothing needs to be stored beyond the index itself – $O(M)$ integers, rather than $O(MD)$ raw patterns or $O(MH)$ engrams.

Table 18. Storage and per-query time complexity ($D=784$, $H=1000$, $L=1000$, T = iteration count).

Method	Storage	Per-query time	Training
Classical/covariance/pseudo-inverse Hopfield	$O(D^2)$, fixed	$O(TD^2)$, fixed	$O(MD^2)$ or $O(M^2D+M^3)$
Modern Hopfield / dense assoc. memory	$O(MD)$, grows	$O(TMD)$, grows	$O(1)$
Sparse distributed memory	$O(LD)$, fixed	$O(LD)$, fixed	$O(MLD)$
k -NN	$O(MD)$, grows	$O(MD)$, grows, no iteration	$O(1)$
Ours	fixed W_{SH} , W_{SI}/W_{IS} + $O(MH)$ growing	$O(TM H)$, grows	$O(M^2D+M^3)$
Vector-HaSH-style modular scaffold	$O(M)$ indices (codes regenerated on demand)	$O(TM \cdot \sum p_i)$, grows	$O(M^2D+M^3)$

The honest summary: neither our architecture nor the modular scaffold escapes the $O(M)$ query-time growth that motivated this whole comparison – both still compare a cue against every stored item, exactly as k -NN, Modern Hopfield, and dense associative memory do. What Vector-HaSH’s actual mechanism contributes beyond our flat scaffold is a smaller, structurally guaranteed storage constant, and a discrimination capacity that is provably decoupled from content dimensionality – not a different complexity class for the content-mapping problem, which remains bounded by the same effective-rank ceiling for every method that must fit a linear map to real images.

4.11 Extending every comparison to $M=4000$

Every result above compares a subset of methods on its own M -grid, generated at different times. As a final consistency and range check, we reran all ten methods – classical/covariance/pseudo-inverse Hopfield, Modern Hopfield, sparse distributed memory, dense associative memory, k -NN, our network, and the Vector-HaSH-style modular scaffold (one-shot and recurrent) – on one identical, fully paired protocol (every method sees the same noisy query on a given trial) spanning $M=100$ –4000, $\sigma=0.3$.

Memory (M) vs. accuracy, all 10 methods, shared Gaussian noise sigma=0.3, N=200/point, fully paired trials

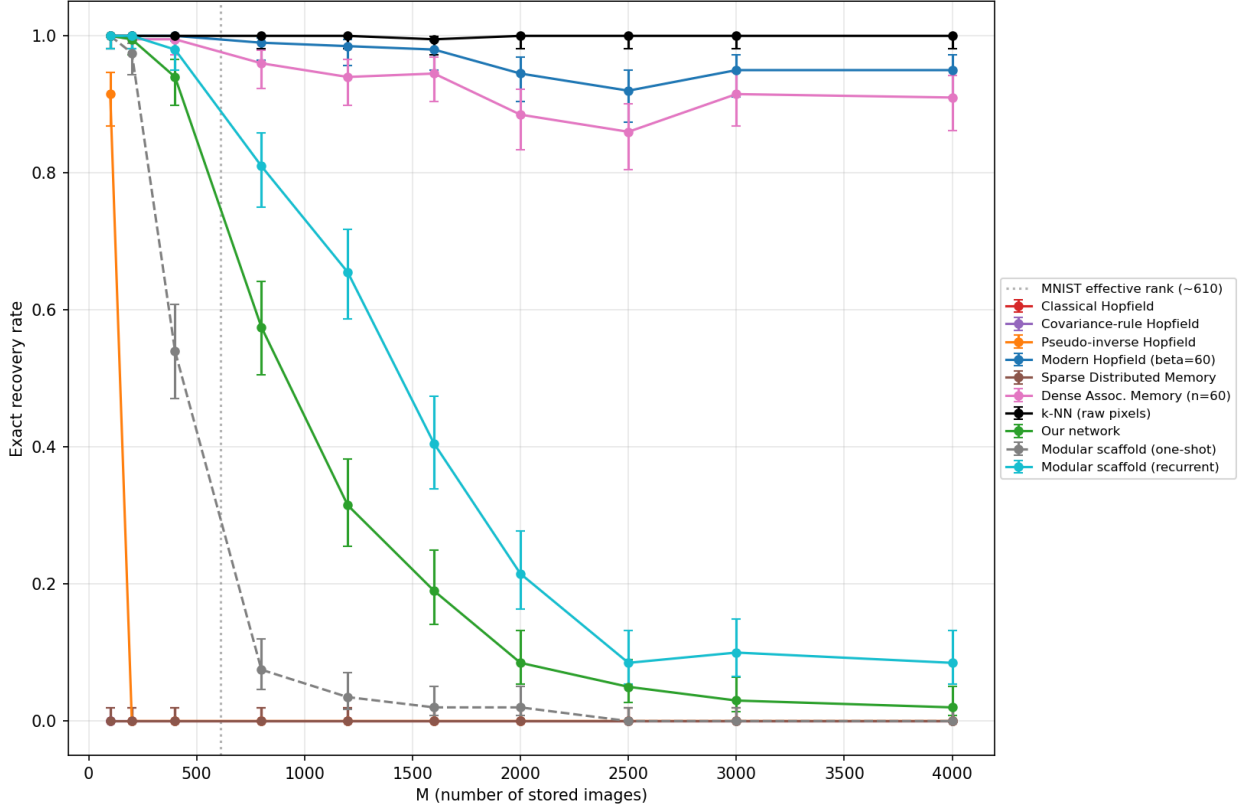


Figure 21. Exact recovery vs. M , all ten methods, fully paired trials ($\sigma=0.3$).

This confirms every number already reported at $M \leq 800$ (Table 14, Figure 15, and Table 17) under a single shared protocol, and extends the picture in one direction not yet shown: Modern Hopfield and dense associative memory remain remarkably robust all the way to $M=4000$ (95.0% and 91.0%), essentially unmoved from their $M=800$ values, while both our own pipeline and the Vector-HaSH recurrent scaffold continue to decline well past $M=800$ – our pipeline to 2.0% and the modular scaffold to 8.5% by $M=4000$. The Vector-HaSH scaffold’s advantage over our own architecture, real and substantial at $M=800$ –2000 (Section 4.10), narrows considerably at this extended range without our pipeline ever catching up: it remains ahead throughout (8.5–10% vs. 2–3% from $M=2500$ to 4000), but the gap is no longer the order-of-magnitude difference seen at $M=800$. k -NN alone shows no decline whatsoever across the entire tested range (100% at every M from 100 to 4000), underscoring the earlier point (Section 4.4) that raw exemplar retrieval, not compressed associative memory of any kind tested here, remains the accuracy ceiling for this specific task.

5 Discussion

Three results compose into a single picture. The combinatorial engram code is robust across a 200-fold range of storage load and independent of connection density (Table 3); the attention-based key-value readout built on top of it shows no observed capacity ceiling where a regression-based readout collapses catastrophically (Table 4); and the symbolic legality constraint this architecture relies on for cleanup can be realized as an emergent property of a spiking network’s phase-synchrony dynamics, statistically matching – not exceeding – the hard-coded rule (Table 8). We regard the

“matches, not exceeds” framing as the honest and more interesting finding: it demonstrates that a biologically-plausible, locally-wired dynamical mechanism can substitute for a piece of hard-coded logic without loss, which is a different and, we think, more useful claim than a performance improvement would have been.

Section 2.3 names why these three results compose rather than merely coexist: the architecture separates sensing (a periodically-driven substrate that never itself reaches equilibrium) from memory (a one-shot combinatorial lookup with no dynamics of its own), with error correction emerging only from the loop coupling them. Classical and modern Hopfield networks make no such separation – sensing and memory are the same fixed-point-seeking dynamical system, which is precisely why their capacity theorems [6] are theorems about that single system’s energy landscape. Ours is a theorem about the memory half alone, derived independently of the spiking substrate and validated against data the spiking substrate had no part in generating (Table 3). That the same combinatorial-memory layer then composes cleanly with a physically-realized dynamical sensor, without degrading the statistical parity result of Table 8, is the concrete payoff of keeping the two separate rather than folding them into one dynamical system from the start.

Two further results test claims we had, until this paper, only argued for by analogy rather than measured directly. First, the encoder itself behaves as a similarity-preserving hash, not merely a randomizing sparsifier: engram overlap tracks raw coincidence-matrix similarity almost monotonically (Spearman $\rho = 0.974$, Section 3.2), the property the fly olfactory hashing circuit [21] is known to provide and that every graceful-degradation curve in this paper implicitly assumes but does not itself demonstrate. Second, unifying correlation-matrix memory, sparse distributed memory, dense associative memory, and Transformer attention as one computation differing only in its separation operator [16] is not just a re-description after the fact – swapping that one nonlinearity in our own readout changes noise robustness by tens of percentage points (Section 3.4), and the mechanism responsible (gap-based vs. ratio-based sharpening under a non-negligible interference floor) is a genuine refinement of, not merely a restatement of, that unification.

A theme recurs across every result in this paper: failure, when it happens, is graceful rather than catastrophic. The combinatorial code and its attention readout hold up to 30–35% query corruption before collapsing over a narrow window rather than failing sharply (Section 3.5); partial-grid completion degrades smoothly with clue count instead of failing at a hard threshold (Table 5); and the spiking mechanism tolerates substantial drive-current noise, remaining a net improvement over the symbolic guess even when corrupted well past the spacing between adjacent digit levels (Section 3.10). Where a cleanup pass exists, its benefit is not uniform but concentrates specifically in the transition region – the conflict-mask cleanup’s rescue effect on query noise peaks sharply at 42% (Figure 4, right) – exactly where ambiguity is highest and correction is most needed, and negligible everywhere else, either because the query was already easy or already unrecoverable. The cross-modal extension (Section 4) reinforces this theme with an external, hard baseline: on identical real images, three classical/pseudo-inverse auto-associative memories all collapse to exactly 0% recovery within a narrow range of stored-item count, while our architecture – sharing the pseudo-inverse learning rule with one of them, but adding a separate legality-based cleanup step – degrades smoothly and remains above 0% an order of magnitude further out (Table 14). This is the same qualitative signature Vector-HaSH’s content-independent scaffold is built to produce, obtained here from a content-*dependent* one.

MESH [12] gives this specific failure mode a name and an explicit cure: it factorizes any content-addressable memory into a fixed, content-independent *scaffold* (their bipartite attractor network; grid-cell/hippocampal recurrence in Vector-HaSH) that alone sets how many states the network can hold, and a separately-learned *heteroassociative* pathway that alone sets how much detail is retrievable about each – producing what they call a CAM continuum: graceful, monotonic loss

of detail rather than sudden loss of everything. Table 14 is, in effect, an independent instance of exactly that contrast, but from a different mechanism: the core engram/readout/cleanup pipeline (Sections 2–3) has no attractor scaffold at all – the engram is a single feedforward hash, not a settled fixed point of any recurrent network – and removing exactly one component (the separate legality-based cleanup step, or, Section 4.7, properly re-validated separation sharpness) is what turns a memory cliff back on. This is consistent with, rather than contradicting, MESH’s diagnosis: what avoids the cliff is not a specific substrate (a physical attractor network, in their case; a symbolic or spiking cleanup step, in ours) but the presence of some mechanism doing more work than raw associative recall alone.

MESH levels a second, distinct criticism at dense associative memory’s bipartite pairwise approximation [12]: achieving its exponential-in- N capacity requires the hidden layer to be resized so that N_{hidden} exactly equals the number of stored memories, so a single fixed network cannot span a range of M – precisely the rigidity their own scaffold is built to avoid. Our own architecture makes an analogous decoupling through a different mechanism. The relational substrate is fixed at $N=81$ Sudoku cells (Section 2.1) and the engram at a fixed $H=1000$ dimensions; neither is ever resized. The identical (N, H, K) triple is used, unmodified, across the entire $M=50$ -to-10,000 range of Table 1 (Section 2.3), where a capacity bound derived once for that fixed triple correctly predicts collapse behavior throughout. Memory capacity here is therefore not a property of the number of relational units or engram dimensions directly – 81 and 1000 are both minuscule – but of the combinatorial space those fixed resources address into, intersected with the actual combinatorial structure of the stored content itself (6.7×10^{21} valid grids, Section 1). This is the same source of capacity MESH’s own scaffold exploits ($\binom{N_L}{k}$ states from N_L label neurons): exponential in fixed resources, rather than requiring those resources to track M . It is worth being precise about what this parallel does *not* claim: like Modern Hopfield’s and dense associative memory’s own pattern matrices (Table 15), our explicit store of (key, value) pairs does grow by one entry per stored item – what stays fixed is the discriminative machinery those keys are drawn from, not the bookkeeping of storing them.

A note on hardware implementability. The comparison above is at the level of statistics, not substrate, but the two families differ in a way worth stating explicitly. MESH and Vector-HaSH’s capacity guarantees rest on pseudoinverse-learned heteroassociative weights ($W_{HF} = HF^+$) – an exact, global computation over the entire stored pattern set that must, in principle, be redone (or approximated online, as MESH’s own Methods discusses) every time a new item is learned, not a purely local, synapse-by-synapse update of the kind neuromorphic hardware is built around. The core engram/readout/cleanup pipeline needs no learned weight matrix at all: W_{SH} is a fixed random projection set once, storage is literally appending one row to a table with no relearning of anything already stored, and the softmax-attention readout has no trainable parameters – a genuine content-addressable memory rather than a weight-matrix-based associative one. The constraint-satisfaction step goes further: its coupling weights (w_{IE}, w_{EE}) are fixed constants gated on and off by a conflict mask computed from the current dynamical state, not learned from data at all (Limitations, below), and it is the only mechanism in this paper’s comparisons already implemented as literal continuous-time spiking neurons rather than a discrete-time $\text{sign}(\cdot)/\text{top-}k$ abstraction. We have not measured this: no energy, latency, or neuromorphic-hardware benchmark was run for any architecture compared in this paper, and the cross-modal extension (Section 4) reintroduces the identical ridge-regularized pseudoinverse rule MESH and Vector-HaSH use, so this argument is specific to the core pipeline, not the full architecture as extended to images. What we can say is architectural rather than measured: the core pipeline’s only nontrivial computation – similarity of a query to M stored keys – is exactly the primitive that in-memory and analog crossbar accelerators compute as a single parallel physical operation, rather than as M sequential comparisons or a

globally-inverted matrix that a pseudoinverse-based scaffold’s write step requires.

We invoke the statistical mechanics of constraint satisfaction and random k -SAT [7, 8] as motivating context for why constraint density and dynamics near a solvability boundary are worth studying together – and, relatedly, why McGuire et al.’s proof that no 16-clue Sudoku has a unique solution [9] is a natural reference point for combinatorial threshold phenomena in this domain – but we do not claim to extend that literature’s universality results. Section 3.7’s honestly reported non-universal exponent is the reason for this restraint, and we consider explicitly reporting a negative/mixed finding here preferable to omitting it.

Limitations. The architecture’s shared, common oscillatory drive is structurally incompatible with anti-phase-based constraint encoding (Section 3.8); this is a property of this specific circuit, not a general claim about spiking constraint satisfaction. The low-clue-fraction regime remains limited by coincidence-matrix specificity, and we have not tested whether a smarter (e.g., adaptive) synaptic mechanism could improve it without compromising the clean matching result of Table 8 – we deliberately did not pursue synaptic plasticity in the reported architecture, since the non-plastic, fixed-weight dynamics already achieve parity with the symbolic rule, and we judged added mechanism without added validated benefit to be the wrong trade for this paper.

The cross-modal extension has its own, separately reported limitations, three of which we tested directly rather than leaving open. First, it is not yet more accurate than a trivial raw-pixel nearest-neighbor classifier at $M=200$ (Section 4.4), and we checked whether this improves at the large- M scale where a fixed-size engram’s storage advantage should start to matter – it does not: the gap widens from +3.5 percentage points at $M=200$ to +96.0 points at $M=2000$ (Table 12), since nearest-neighbor accuracy is nearly insensitive to M while ours is not. Whatever this architecture is for, it is not for beating nearest-neighbor lookup at any M we tested; its case would have to rest on properties outside classification accuracy entirely (e.g. fusing partial pixel and partial relational cues, which nearest-neighbor over images alone cannot do). Second, the ridge-regularization question is resolved rather than open: extending the sweep to $M=4000$ (Table 13) shows the required λ does *not* keep growing – a single $\lambda \approx 10$ stays near-optimal throughout – and the residual decline past $M=2000$ is fully attributable to the already-identified image effective-rank ceiling, not to unresolved regularization. Third, the Hopfield comparison’s mismatched noise models (bit-flip vs. Gaussian) have been replaced with a single shared model (Section 4.6), which revealed a more precise finding than the original qualitative one: at fixed, small M our network and the pseudo-inverse baseline are statistically comparable in noise tolerance – the real advantage is capacity scaling, not noise robustness per se. That advantage, however, does not generalize to the broader associative-memory literature: extending the comparison to Modern Hopfield networks and dense associative memory (Section 4.7), both given properly re-validated separation sharpness rather than naively carried-over defaults, shows both substantially outperform our raw recovery accuracy at high M under matched noise (99.0% and 96.0% vs. our 57.5% at $M=800$, $\sigma=0.3$), a gap that a $3\times$ larger engram narrows by about half but does not close. We report this as a genuine limitation, not a caveat to explain away: this architecture’s case cannot rest on being the best available image-associative memory. It rests instead on a structural property neither alternative has – the key (a sparse engram drawn from an astronomically large combinatorial space, decoupled from the content’s own representation) is entirely separate from the value (arbitrary content, here images, but Section 4.3 shows three unrelated content types share the identical key infrastructure, which simultaneously performs the Sudoku-legality task of Sections 2–3). Where the two kinds of claim actually overlap – recovery at low-to-moderate noise – our architecture already matches both purpose-built alternatives.

Is the β comparison itself fair? Section 4.7’s Tuning transparency paragraph sets Modern Hopfield’s $\beta=60$ against our own $\beta=1/\tau=0.5$ by validating each separately for zero-noise self-

recovery, not by matching the raw numbers – but the two readouts are the identical functional form ($p = \text{softmax}(\beta \cdot \text{sim})$, our $\tau=2.0$ giving $\beta=0.5$), so it is worth asking directly whether a $120\times$ smaller β secretly biases the headline gap above in Modern Hopfield’s favor. Raw β is not comparable across models: it multiplies a similarity score living on a completely different scale in each case – Modern Hopfield’s is a cosine similarity in $[-1, 1]$ over correlated raw pixels, ours is an integer engram-overlap count in $[0, K=150]$ – so the quantity that actually sets discriminative sharpness, per Ramsauer et al.’s own separation theorem, is the product $\beta \cdot \Delta$, where Δ is the mean gap between a query’s true-match similarity and its best competitor. We measured Δ directly for both models, at clean queries, across the full $M=100\text{--}4000$ sweep already used for the Modern Hopfield cliff test (Section 4.7):

Table 19. Effective separation sharpness $\beta \cdot \Delta$, clean queries, $n=300$ sampled queries/point.

M	MH Δ	MH $\beta\Delta$	Ours Δ	Ours $\beta\Delta$	Ratio (ours/MH)
100	0.264	15.8	106.8	53.4	$3.4\times$
200	0.232	13.9	105.6	52.8	$3.8\times$
400	0.208	12.5	101.7	50.8	$4.1\times$
800	0.190	11.4	78.9	39.4	$3.5\times$
1200	0.176	10.6	57.9	29.0	$2.7\times$
1600	0.168	10.1	45.5	22.7	$2.3\times$
2000	0.162	9.7	35.9	18.0	$1.9\times$
2500	0.168	10.1	28.6	14.3	$1.4\times$
3000	0.160	9.6	20.4	10.2	$1.1\times$
4000	0.146	8.7	13.7	6.8	$0.8\times$

At the $M \leq 800$ range where the headline comparison (Table 15, Figure 15) is actually drawn, our effective sharpness is $3.4\text{--}4.1\times$ larger than Modern Hopfield’s, not smaller – so the reported 57.5% vs. 99.0% gap at $M=800$, $\sigma=0.3$ is, if anything, generous to Modern Hopfield, not an artifact of an unfairly weak β on our side. But the advantage is not stable across M : it shrinks monotonically and crosses parity near $M \approx 3000$, and by $M=4000$ Modern Hopfield’s effective sharpness is slightly the larger of the two (8.7 vs. 6.8). This crossover has a precise mechanistic source, not a tuning artifact. Modern Hopfield’s Δ is nearly flat with M ($0.264 \rightarrow 0.146$, less than $2\times$) because cosine similarity between raw, 784-dimensional pixel vectors is governed by concentration of measure and barely moves as more images are added; ours collapses more than 7-fold ($106.8 \rightarrow 13.7$) because it is set not by the readout but by the upstream pseudo-inverse encoding step, whose fidelity is capped by MNIST’s effective rank ($\sim 600\text{--}610$, Section 4.5) – once M approaches and passes that ceiling, every stored engram becomes a slightly worse fit, competitor overlaps rise, and Δ falls. This gives the high- M gap reported in Section 4.7 a specific cause rather than only an empirical description: it is the same encoding-stage bottleneck already identified in Section 4.5, now visible directly in the separation term itself rather than only in downstream accuracy, and it is symmetric with – not an artifact of – the β choice on either side.

Two kinds of error correction. Every alternative memory tested in Sections 4.6–4.7 corrects errors by an abstract mathematical procedure: classical/covariance/pseudo-inverse Hopfield and dense associative memory descend an energy function via synchronous sign updates; Modern Hopfield iterates a softmax fixed-point map; a symbolic Sudoku solver would backtrack over candidate assignments. None of these is a physical process – each requires a global objective (an energy, a normalized similarity distribution, a search tree) that the update rule has privileged access to. Sections 3.8–3.9 of this paper test a genuinely different kind of mechanism for the same job: a network of leaky-integrate-and-fire neurons, coupled only through local excitatory/inhibitory synapses with no encoded objective function at all, whose relaxation to a phase-synchronized steady state realizes the identical legality-checking computation – verified statistically indistinguishable from the hard-coded rule it replaces (Table 8). This is not a claim we extend to the image-heteroassociation comparison above, where the cleanup step is the symbolic conflict-mask, not the spiking network

(Section 4.7 notes this explicitly) – but it is a validated property of this same architecture’s core task, and none of the six alternative memories in Sections 4.6–4.7 attempt in any form.

The scaffold, not just the learning rule, is a free choice. Section 4.10 tests Vector-HaSH’s actual modular addressing mechanism, not just the heteroassociation rule borrowed from it, and finds it beats our own flat scaffold once given a genuinely recurrent (not independent) cleanup step – an instructive result against our own architecture, obtained by taking a cited comparison seriously enough to build rather than only invoke. A complexity analysis sharpens what this actually buys: neither the modular scaffold nor our own engram escapes $O(M)$ query-time growth, the same class k -NN and the sharp associative memories of Section 4.7 already occupy – the real, structural advantage modularity contributes is a discrimination capacity provably decoupled from content dimensionality, plus an $O(M)$ -index storage cost rather than an $O(MD)$ or $O(MH)$ one. Content fidelity itself remains bounded by the same image effective-rank ceiling as everything else. We view this as strengthening, not undercutting, the honest accounting in Section 4.7: this architecture’s case was never that it is the most efficient or most accurate available memory, but that its specific combination of properties – a fixed key infrastructure serving arbitrary content, a combinatorially large native address space, and (on its original task) a physically-realizable dynamical error-correction mechanism – is not available from any single alternative tested.

6 Methods

Section 2.3’s expected-maximum bound (Table 1) is computed by `capacity_bound_derivation.py` – an analytical calculation, not a simulation: exact hypergeometric moments for the two similarity distributions, the standard leading-order Gaussian extreme-value approximation for the maximum of $M-1$ competitors, and a numerical solve of $M_{\max}(\eta) = M$ over a fine grid of η . The rigorous exact hypergeometric tail bound (Table 2) is computed separately by `capacity_bound_exact_tail.py`: no Gaussian approximation anywhere, using `scipy.stats.hypergeom` directly for both the corruption distribution R and the per-competitor tail probabilities $p_k(s)$, with the numerically stable form $1 - (1-p)^n = -\text{expm1}(n \cdot \log1p(-p))$ used throughout to avoid catastrophic cancellation at the very small probabilities involved.

All simulations use Brian2 [10]. Spiking network parameters: $\tau_m = 40$ ms, $f_{\text{osc}} = 10$ Hz, $V_{\text{th}} = 1$, $V_{\text{reset}} = 0$, integration timestep 0.1 ms (RK4), $w_{IE} = 0.005$, $w_{EE} = 0.001$ (Section 3.8 operating point). Digit-encoding drive currents use nine evenly spaced levels chosen so every level reliably crosses threshold at least once per oscillation cycle (below ~ 0.46 , a level never fires; above this floor, separation in first-spike phase is monotonic in current). Simulation window 3000 ms per trial, with the first 1500 ms discarded as transient; every trial builds a fresh network instance (Brian2’s absolute simulation clock does not reset on a reused network object, so a persistent network across trials would confound successive trials’ oscillation phase). Initial membrane potential is randomized per trial (Uniform(0, 0.1) + Uniform(−0.1, 0.1) perturbation).

Engram/readout parameters: $H = 1000$, $K = 150$ (capacity/robustness sweeps) or $K = 400$ (crossover characterization, chosen as the sharpest transition in an earlier M – K grid search), projection density $\rho = 0.02$, softmax temperature $\tau = 2.0 \times (K/150)$ (scaled to the engram’s self-overlap). Statistical tests: two-sided Wilcoxon signed-rank (`scipy.stats.wilcoxon`) on paired per-trial accuracy differences, identical target grid and known-cell mask across compared methods within each trial.

Heteroassociation (Section 4): W_{SI}/W_{IS} trained via the dual-form ridge pseudo-inverse ($\lambda = 10^{-4}$ default, $\lambda = 0.1$ – 10 where noted for larger M , verified stable up to $M=4000$), images normalized to $[0, 1]$ and Gaussian noise added and reclipped to that range. MNIST/Fashion-MNIST are

28×28 (dimension 784); CIFAR-100 is converted to grayscale by averaging RGB channels (32×32 , dimension 1024). Baseline (Section 4.4): paired McNemar exact test (`scipy.stats.binomtest` on discordant-pair counts) between our pipeline’s and nearest-neighbor’s exact-match outcome on identical noisy queries, at $M=200$ and, separately, across $M=200\text{--}2000$ at fixed $\sigma=0.4$. Hopfield baselines (Section 4.6): images bipolarized by thresholding at 0.5; classical rule is the outer-product/Hebbian sum with zero diagonal; covariance rule subtracts each pixel’s own mean activity (not a single global scalar) before the same sum; pseudo-inverse rule reuses the identical dual-form solver as W_{SI}/W_{IS} with the pattern set as both input and target. Synchronous recall update $s \leftarrow \text{sign}(Ws)$ preserves a neuron’s current state on an exact tie (net input 0) rather than forcing it to +1, since mean-centered patterns can have exactly-zero weights on pixels that are constant across every stored image. The headline Hopfield comparison (Table 14) uses a single shared noise model across all four methods: a Gaussian-corrupted continuous image is generated once per trial, consumed directly by our network and bipolarized (thresholded at 0.5) for the three Hopfield variants, at $n=200/\text{point}$ with Wilson 95% CIs; an earlier bit-flip-vs-Gaussian mismatched version is reported separately in the text as a mechanism illustration only.

Modern Hopfield/SDM/DAM comparison (Section 4.7): Modern Hopfield patterns are L_2 -normalized (both stored patterns and the query) before the softmax update $\xi \leftarrow X^\top \text{softmax}(\beta X \xi)$, iterated to a fixed point (max 20 iterations, $\beta=60$); success is nearest-stored-pattern correctness by Euclidean distance, not exact numerical equality, since a converged continuous state need not match bit-for-bit. Dense associative memory uses the synchronous update $s \leftarrow \text{sign}(\sum_\mu n \cdot \text{ReLU}(s \cdot x_\mu/d)^{n-1} x_\mu)$, $n=60$, iterated to convergence (max 30 iterations). Sparse distributed memory uses $L=1000$ random bipolar hard locations and a Hamming radius chosen so a pair of independent random bipolar vectors of the stored dimension ($d=784$) falls within it with probability 0.02 (via the binomial quantile function); writing accumulates the full stored pattern into every hard location within radius, reading sums the counters of every activated hard location and thresholds by sign. All three methods’ sharpening parameters (β , n , and SDM’s target activation fraction) were validated by checking zero-noise self-recovery across the full $M=100\text{--}800$ sweep before any noise-robustness number was computed (Section 4.7); naive carryovers from other contexts in this paper ($\beta=0.5$, $n=8$) fail this check almost completely. The H -scaling test (Table 16) reuses the identical pipeline as Section 4.5 with H varied and $K=0.15H$ held fixed throughout.

$\beta \cdot \Delta$ fairness check (Table 19): for Modern Hopfield, Δ is the mean, over 300 sampled stored patterns per M , of the true pattern’s cosine self-similarity (exactly 1.0 under L_2 -normalization) minus its highest-similarity competitor among the remaining $M-1$ stored patterns. For our network, Δ is the mean, over the same sample size, of the true engram’s overlap with the query-reconstructed engram ($H_{\text{keys}} \cdot h_{\text{query}}$, the same quantity fed to the readout softmax) minus the highest-overlap competitor, using clean ($\sigma=0$) queries throughout so the measurement isolates representational geometry rather than noise-driven distortion.

Appendix: M , K , H by Figure

H (hippocampal/engram dimension) and K (active units per engram) are 1000 and 150 throughout this paper unless noted otherwise – never tuned per figure. M (number of stored items) is the one parameter that varies, either held fixed or swept as the figure’s own x -axis. Table 20 gives the exact values for every figure, read directly from each figure’s generating script rather than reconstructed from memory.

Table 20. M , K , H for every figure in this paper.

Fig.	Generating script	M	K	H
1	hash_similarity.preservation.py	200 (30 targets sampled)	150	1000
2	capacity_regression.vs.attention.py	10–2000 (swept, x -axis)	150	1000
3	separation.operator.ablation.py	200	150	1000
4	hippocampus.feedback.noise.zoomed.py	200	150	1000
5	sudoku.partial.grid.completion.py	2000	150	1000
6	critical.slowing.down.clean.plot.py	500	400	1000
7	spiking.real.sudoku.sweetspot.py	–	–	–
8	spiking.emergent.coincidence.py	200	150	1000
9	spiking.drive.noise.robustness.py	200	150	1000
10	heteroassoc.vectorhash.best.examples.py	200	150	1000
11	heteroassoc.digit.classifier.seed.sweep.py	see note	150	1000
12	heteroassoc.digit.classifier.vs.baseline.confusion.py	see note	150	1000
22	heteroassoc.digit.classifier.best.worst.py	see note	150	1000
13	heteroassoc.final.consolidation.py	see note	150	1000
14	hopfield.matched.noise.rigorous.py	see note	150	1000
15	hopfield.modern.sdm.dam.comparison.py	see note	150	1000
16	hopfield.comparison.followup.H.and.highM.noise.py	see note	150	1000
17	hopfield.comparison.followup.H.and.highM.noise.py	800	see note	see note
20	final.hetero.comparison.plot.py (see note)	100–800 (swept, x -axis)	n/a	n/a

Figure 7 is a single-grid spiking simulation ($N=81$ cells) with no engram storage or retrieval involved, so $M/K/H$ do not apply. Figure 6 uses $K=400$ – the sharpest crossover found in an earlier M – K grid search – rather than the default 150. Figures 3 and 4 both sweep query noise on paired corrupted queries, but differ in what is held fixed: Figure 4 fixes the separation operator to softmax and includes the conflict-mask cleanup pass; Figure 3 instead varies the separation operator itself, with cleanup excluded throughout, to isolate the operator’s own behavior from the cleanup mechanism characterized in Figure 4. Figures 11, 12, and 22 (the digit classifier’s confusion structure, baseline comparison, and example classifications) do not store Sudoku grids in the usual sense: $M=9$ class-target engrams are trained against, each regressed from 500 training images/class (see the Appendix, “Digit Classifier Train/Test Set and Example Classifications,” for the exact split), drawn from an $M_{\text{pool}}=200$ -grid Sudoku pool that the 9 targets are sampled from. Figures 13 and 14 each have panels at different M : Figure 13’s top-left (pipeline vs. nearest-neighbor) and top-right/bottom-left (capacity cliff and Gram-matrix conditioning) panels use $M=200$ and $M \in \{200, 300, 400, 500, 650, 800\}$ respectively, and its bottom-right (ridge sensitivity) panel fixes $M=500$; Figure 14’s left panel (M -sweep) uses $M \in \{100, 200, 400, 800\}$ at fixed $\sigma=0.3$, and its right panel (noise-sweep) fixes $M=50$. Figure 15 follows the identical M/σ grid as Figure 14 (left panel $M \in \{100, 200, 400, 800\}$ at $\sigma=0.3$; right panel $M=50$ noise-sweep), extended to three further methods (Modern Hopfield, sparse distributed memory, dense associative memory) that store the full 784-dimensional image directly rather than an H/K -parameterized engram, so K/H apply only to the “ours” curve in each panel. Figure 16 repeats the same right-panel noise-sweep at $M=50$ (left) and $M=800$ (right), for the same seven methods. Figure 17 sweeps $H \in \{1000, 1500, 2000, 3000\}$ ($K=0.15H$ throughout) for “ours” only, at fixed $M=800$, $\sigma=0.3$; the two dashed reference lines are Modern Hopfield’s and dense associative memory’s $H=1000$ -comparison values from Figure 15, not additional swept points. Figure 20 uses a 10-module modular grid code ($\sum p_i=1164$ total units, 10 active per code) in place of an H/K -parameterized engram, so those columns are marked n/a rather than reusing the default 150/1000. Its data is produced by `vectorhash_modular_scaffold_comparison.py` and plotted by `final_hetero_comparison_plot.py`.

Appendix: Digit Classifier Train/Test Set and Example Classifications

The digit classifier of Sections 4–4.2 is trained and evaluated on MNIST digits 1–9 (digit 0 excluded, to match the nine Sudoku digit values used as class targets). Table 21 gives the exact split. Train and test images are disjoint subsets of Keras’s standard MNIST train/test partitions respectively (`tensorflow.keras.datasets.mnist`), so no test image is seen during training; within each partition, the first N images of each class are taken in dataset order (deterministic, not randomly sampled), via `np.flatnonzero(y==d)[:n]`.

Table 21. Train/test set construction for the digit classifier.

Dataset	MNIST (Keras standard train/test partitions)
Classes	9 (digits 1–9; digit 0 excluded)
Train images per class	500
Test images per class	100
Total train images	4500
Total test images	900
Image dimension	$28 \times 28 = 784$ (flattened, normalized to $[0, 1]$)
Train/test overlap	none (disjoint Keras partitions)

Figure 22 shows, for every one of the 9 classes, the held-out test image classified with the single highest margin (top-scoring class minus runner-up) among correct classifications (“best”) and the held-out test image classified with the highest margin among *incorrect* classifications (“worst”: confidently wrong), at zero query noise. Digit 1’s worst case is a genuine low-margin ambiguity (0.41, well below every other class’s ~ 0.7 – 0.8); every other class’s worst case is a high-margin error, consistent with the attractor structure quantified in Section 4.2.

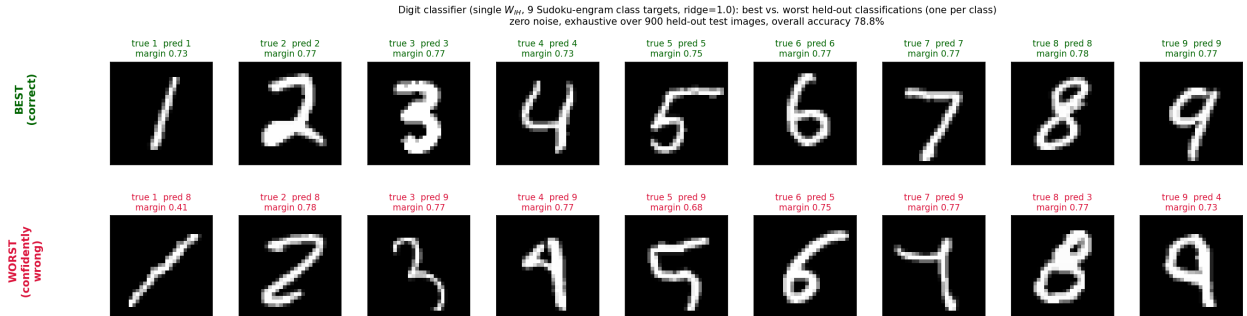


Figure 22. One best and one worst held-out classification per digit class (zero noise, exhaustive over all 900 test images).

Code and Data Availability

All simulation code, raw data (CSV), and figure-generation scripts are available at <https://github.com/ghub-res1578/sudoku-hippocampal-key-value-memory>, including a README with exact commands to regenerate every table and figure in this paper.

References

- [1] D. Marr, “Simple memory: a theory for archicortex,” *Phil. Trans. R. Soc. Lond. B* **262**, 23–81 (1971).
- [2] E. T. Rolls, “The mechanisms for pattern completion and pattern separation in the hippocampus,” *Front. Syst. Neurosci.* **7**, 74 (2013).
- [3] J. J. Hopfield, “Neural networks and physical systems with emergent collective computational abilities,” *PNAS* **79**, 2554–2558 (1982).
- [4] D. J. Willshaw, O. P. Buneman, H. C. Longuet-Higgins, “Non-holographic associative memory,” *Nature* **222**, 960–962 (1969).
- [5] D. Krotov, J. J. Hopfield, “Dense associative memory for pattern recognition,” *NeurIPS* (2016).
- [6] H. Ramsauer et al., “Hopfield networks is all you need,” *arXiv:2008.02217* (2020).
- [7] M. Mézard, G. Parisi, R. Zecchina, “Analytic and algorithmic solution of random satisfiability problems,” *Science* **297**, 812–815 (2002).
- [8] M. Mézard, A. Montanari, *Information, Physics, and Computation*, Oxford University Press (2009).
- [9] G. McGuire, B. Tugemann, G. Civario, “There is no 16-clue Sudoku: solving the Sudoku minimum number of clues problem via hitting set enumeration,” *Experimental Mathematics* **23**, 190–217 (2014).
- [10] M. Stimberg, R. Brette, D. F. M. Goodman, “Brian 2, an intuitive and efficient neural simulator,” *eLife* **8**, e47314 (2019).
- [11] S. Chandra, S. Sharma, R. Chaudhuri, I. Fiete, “Episodic and associative memory from spatial scaffolds in the hippocampus,” *Nature* **638**, 739–752 (2025).
- [12] S. Sharma, S. Chandra, I. Fiete, “Content addressable memory without catastrophic forgetting by heteroassociation with a fixed scaffold,” *ICML* (2022).
- [13] L. Personnaz, I. Guyon, G. Dreyfus, “Collective computational properties of neural networks: New learning mechanisms,” *Physical Review A* **34**(5), 4217 (1986).
- [14] I. Kanter, H. Sompolinsky, “Associative recall of memory without errors,” *Physical Review A* **35**(1), 380 (1987).
- [15] J. L. McClelland, B. L. McNaughton, R. C. O’Reilly, “Why there are complementary learning systems in the hippocampus and neocortex: insights from the successes and failures of connectionist models of learning and memory,” *Psychol. Rev.* **102**, 419–457 (1995).
- [16] S. J. Gershman, I. Fiete, K. Irie, “Key-value memory in the brain,” *Neuron* **113**, 1694–1707 (2025).
- [17] T. Kohonen, “Correlation matrix memories,” *IEEE Trans. Comput.* **C-21**, 353–359 (1972).
- [18] P. Kanerva, *Sparse Distributed Memory*, MIT Press (1988).

- [19] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, I. Polosukhin, “Attention is all you need,” *NeurIPS* **30** (2017).
- [20] I. Schlag, K. Irie, J. Schmidhuber, “Linear Transformers Are Secretly Fast Weight Programmers,” *ICML* (2021).
- [21] S. Dasgupta, C. F. Stevens, S. Navlakha, “A neural algorithm for a fundamental computing problem,” *Science* **358**, 793–796 (2017).
- [22] C. von der Malsburg, “The correlation theory of brain function,” Internal Report 81-2, Max Planck Institute for Biophysical Chemistry (1981).
- [23] W. Singer, C. M. Gray, “Visual feature integration and the temporal correlation hypothesis,” *Annu. Rev. Neurosci.* **18**, 555–586 (1995).
- [24] J. J. Hopfield, D. W. Tank, “Neural computation of decisions in optimization problems,” *Biol. Cybern.* **52**, 141–152 (1985).